

# Gaussian Processes for Machine Learning

*Ross Bathgate, Colin McDonagh, Jack  
Taylor, Yifei Wang, Yuqing Zhao*

Year 4 Project  
School of Mathematics  
University of Edinburgh  
2024 - 2025

# Abstract

This report examines Gaussian Processes (GPs) in machine learning, focusing on regression, classification, training point selection and optimisation. We introduce three applications: a tailored kernel design to model domain-specific patterns, GP regression for analysing noisy snowline data, and probabilistic classification of heart disease using approximation methods to quantify uncertainty. We also compare strategies for selecting training points to improve efficiency and propose a Bayesian optimisation framework that updates hyperparameters dynamically. Through theoretical analysis and case studies, this work demonstrates how GPs balance interpretability and performance in real-world scenarios.

# Declaration

I declare that this thesis was composed by myself and that the work contained therein is my own, except where explicitly stated otherwise in the text.

*(Ross Bathgate, Colin McDonagh, Jack Taylor, Yifei Wang, Yuqing Zhao)*

*We'd like to thank Professor Aretha Teckentrup for her invaluable guidance and support throughout our work, as well as our families and friends for their constant encouragement.*



# Contents

|                                                                                  |           |
|----------------------------------------------------------------------------------|-----------|
| <b>Abstract</b>                                                                  | <b>ii</b> |
| <b>Contents</b>                                                                  | <b>vi</b> |
| <b>1 Introduction</b>                                                            | <b>1</b>  |
| 1.1 Background . . . . .                                                         | 1         |
| 1.2 Objectives of the Report . . . . .                                           | 1         |
| <b>2 GP Theory and Foundations</b>                                               | <b>3</b>  |
| 2.1 Introduction to GPs . . . . .                                                | 3         |
| 2.2 Choice of Kernels . . . . .                                                  | 6         |
| 2.2.1 Examples of Covariance Functions . . . . .                                 | 6         |
| 2.2.2 Constructing New Kernels From Known Kernels . . . . .                      | 10        |
| 2.2.3 Example of a Custom Kernel . . . . .                                       | 11        |
| <b>3 GP Regression</b>                                                           | <b>14</b> |
| 3.1 Theory of GP Regression . . . . .                                            | 14        |
| 3.2 Example: Snowline Dataset Regression . . . . .                               | 16        |
| 3.2.1 Dataset & Kernel Choice . . . . .                                          | 16        |
| 3.2.2 GP Predictions . . . . .                                                   | 17        |
| 3.2.3 Limitations . . . . .                                                      | 18        |
| 3.2.4 An Alternative Approach and Comparison to a Meteorological Model . . . . . | 19        |
| 3.3 Impact of Kernel Choice . . . . .                                            | 20        |
| <b>4 GP Classification</b>                                                       | <b>22</b> |
| 4.1 Theory of GP Classification . . . . .                                        | 22        |
| 4.2 Application to Heart Disease Dataset . . . . .                               | 26        |
| 4.2.1 Data Analysis . . . . .                                                    | 26        |
| 4.2.2 Applying the Laplace Approximation . . . . .                               | 27        |
| <b>5 Training Point Selection for GPs</b>                                        | <b>29</b> |
| 5.1 One-shot Training Point Selection . . . . .                                  | 29        |
| 5.1.1 Latin Hypercube Sampling . . . . .                                         | 30        |
| 5.1.2 Sobol Sequences . . . . .                                                  | 30        |
| 5.1.3 Comparing One-Shot Selection Methods . . . . .                             | 31        |

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| 5.2      | Sequential Training Point Selection . . . . .              | 32        |
| 5.2.1    | Active Learning MacKay . . . . .                           | 33        |
| 5.2.2    | Evaluating the Performance of ALM . . . . .                | 34        |
| <b>6</b> | <b>Bayesian Optimisation</b>                               | <b>36</b> |
| 6.1      | The Bayesian Approach to Model Selection: . . . . .        | 36        |
| 6.1.1    | Model Selection . . . . .                                  | 36        |
| 6.1.2    | Hyperparameter Selection for GPs . . . . .                 | 37        |
| 6.2      | Theory of Bayesian Optimisation . . . . .                  | 40        |
| 6.2.1    | Constructing the Surrogate Model . . . . .                 | 40        |
| 6.2.2    | Defining and Maximising the Acquisition Function . . . . . | 41        |
| 6.2.3    | Iterative Optimisation Process . . . . .                   | 43        |
| 6.3      | Adaptive Bayesian Optimisation . . . . .                   | 44        |
| 6.3.1    | Optimising a Noisy 1D Unknown Function . . . . .           | 46        |
| 6.3.2    | Joint Hyperparameter Optimisation for a GP . . . . .       | 48        |
| <b>7</b> | <b>Conclusion</b>                                          | <b>50</b> |
| 7.1      | Summary of Findings . . . . .                              | 50        |
| 7.2      | Future Work . . . . .                                      | 50        |
|          | <b>Bibliography</b>                                        | <b>52</b> |
| <b>A</b> | <b>Code</b>                                                | <b>55</b> |
| A.1      | Creating a Custom Kernel Code . . . . .                    | 55        |
| A.2      | Snowline Regression Analysis Code . . . . .                | 59        |
| A.3      | Heart Disease Application Code . . . . .                   | 69        |
| A.4      | Active Learning Experiment . . . . .                       | 73        |
| A.5      | Adaptive Bayesian Optimisation Code . . . . .              | 76        |
| <b>B</b> | <b>Figures</b>                                             | <b>80</b> |
| B.1      | Adaptive Bayesian Optimisation 2D Updates . . . . .        | 80        |

# Chapter 1

## Introduction

GPs are a powerful Bayesian approach that models distributions over functions rather than relying on parametric forms. By capturing complex patterns through kernel functions and providing uncertainty estimates, they enable more robust decision-making in tasks such as regression, classification, and optimisation.

### 1.1 Background

Bayesian inference provides a framework for updating prior beliefs with new data. Initially, Bayesian methods were applied to parametric models [1]. However, as real-world problems grew more complex, the focus shifted toward nonparametric methods that allow the model complexity to increase with the available data.

In machine learning, GPs have become a valuable tool due to their flexibility and capacity to quantify uncertainty in predictions. The work of Rasmussen and Williams [1] demonstrated that GPs could be applied to regression, classification, and structured prediction tasks by capturing complex, non-linear relationships without the need to assume a fixed functional form. Their nonparametric nature enables GPs to adapt to the data, which means they can adjust as new patterns emerge, even in high dimensions. This methodology has been successfully applied in various domains, including time-series forecasting and spatial modelling [2] [3].

### 1.2 Objectives of the Report

This report aims to provide a concise yet comprehensive overview of GPs in machine learning, focusing on their applications in regression, classification, and optimisation. By developing a rigorous theoretical foundation and validating it with practical examples, this work seeks to outline existing knowledge and introduce meaningful improvements. Our key objectives are as follows:

- **Demonstrate Novel Applications of GPs:** We explore GPs in diverse domains by applying them to challenging datasets. This includes:
  - **Custom Kernel Design:** Developing and implementing a tailored kernel to show how kernel choice influences model behavior and performance.
  - **Snowline Regression:** Applying GP regression to a snowline dataset to examine the ability of GPs to model noisy and spatially varying environmental data.
  - **Heart Disease Classification:** Using GP classification with Laplace approximation and a baseline model on a heart disease dataset to demonstrate the potential of GPs when supporting medical decisions.
- **Investigate Training Point Selection Strategies:** We systematically compare different training point designs for GP models, demonstrating that targeted sampling improves model generalisation and performance only when the underlying function allows it.
- **Enhance Bayesian Optimisation Methods:** Recognising the computational challenges of costly function evaluations, we propose improvements to the standard Bayesian optimisation framework. Our approach incorporates adaptive updates of surrogate hyperparameters during the optimisation process, enabling the model to adjust dynamically and converge more rapidly toward a global optimum.

# Chapter 2

## GP Theory and Foundations

### 2.1 Introduction to GPs

GPs are essentially a generalisation of the multivariate Gaussian distribution. For completeness, we recall the definitions of a positive (semi-) definite matrix, and a multivariate Gaussian distribution here.

**Definition 1.** [4] A real  $n \times n$  matrix,  $K$ , that satisfies  $\mathbf{v}^\top K \mathbf{v} \geq 0$  for all  $\mathbf{v} \in \mathbb{R}^n$  is said to be positive semi-definite. If equality holds if and only if  $\mathbf{v} = \mathbf{0}$ , then  $K$  is said to be positive definite.

**Definition 2.** [5] A multivariate Gaussian distribution over  $n$  random variables is specified by a mean vector  $\boldsymbol{\mu}$  and a positive semi-definite covariance matrix  $\boldsymbol{\Sigma}$  of dimensions  $n \times 1$ ,  $n \times n$  respectively. These parameters are related through the density function,

$$\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) := \frac{1}{(2\pi)^{n/2} |\boldsymbol{\Sigma}|^{1/2}} \exp \left( -\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right).$$

Figure 2.1 compares samples drawn from multivariate Gaussians in 2 dimensions with different covariance matrices, and mean  $\mathbf{0}$ .

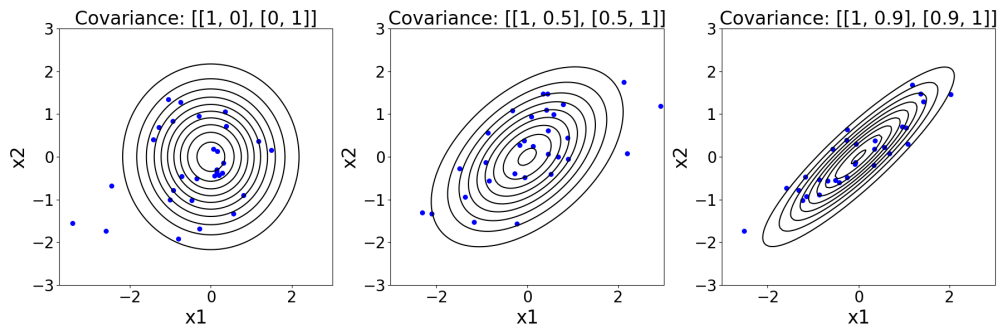


Figure 2.1: Contour plots of multivariate Gaussians, each with 30 samples drawn.

Let us now consider just an individual sample (one of the blue dots in Figure 2.1). Instead of plotting in a 2-dimensional plane as above, consider the plot in Figure 2.2 (*left*), where we plot the sample point  $\mathbf{x} = (x_1, x_2)$  by separating the  $x_1$  and  $x_2$  components. We can generalise this idea by increasing the dimensionality of the sample, shown in the *middle* and *right* plots. Notice how that, as we increase the dimensionality, the values of each variable appear to follow some smooth “function” of the variables. This function represents a single sample of the multivariate Gaussian, and different samples would generate different functions. The reason the values change in a smooth manner comes down to the covariance matrix  $\Sigma$  used to generate the samples. Elements in  $\Sigma$  were generated according to an *RBF kernel* (more on kernels later in Section 2.2) which creates higher correlation between values that are close to each other. For example, looking at Figure 2.2 (*right*) the value of variable 2 is highly correlated with the value of variable 1, but there is almost no correlation with variable 25. The covariance matrix for  $n = 50$  generated using the RBF kernel is shown using a heatmap in Figure 2.3.

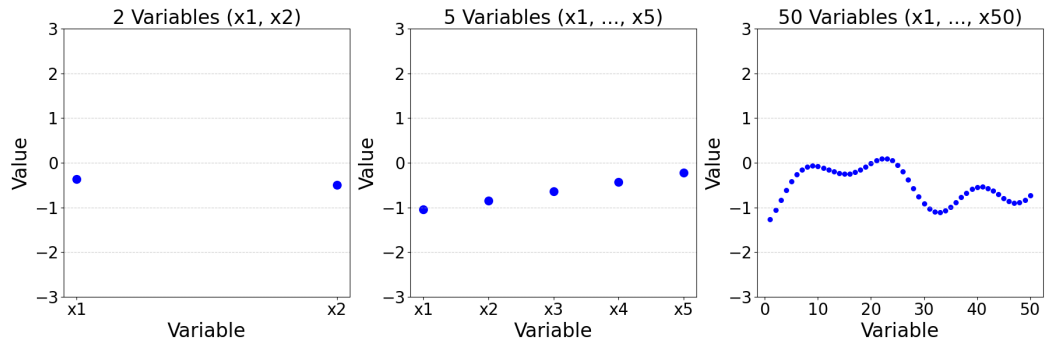


Figure 2.2: (*Left*) Single sample from a 2-dimensional Gaussian with variable labels on the x axis and corresponding values on the y axis. (*Middle*) Single sample from a multivariate Gaussian with  $n = 5$ . (*Right*) Single sample from a multivariate Gaussian with  $n = 50$ .

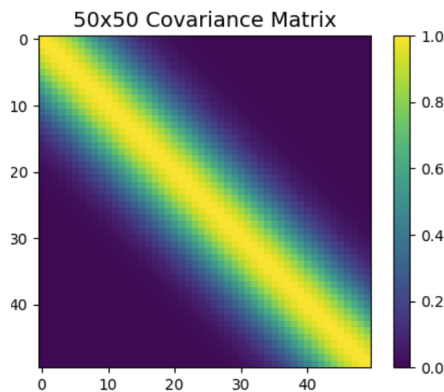


Figure 2.3: Covariance matrix for  $n = 50$  using RBF kernel.

Naturally, the samples we take from the multivariate Gaussian have some variance. This results in different functions each time we sample, as illustrated in Figure 2.4, where 20 samples are taken from a multivariate Gaussian over 50

variables. We have connected the dots with lines to help identify the individual samples. The multivariate Gaussian had mean zero, so we expect the mean sample, or function, to be approximately zero for all variables, as is the case for these samples.

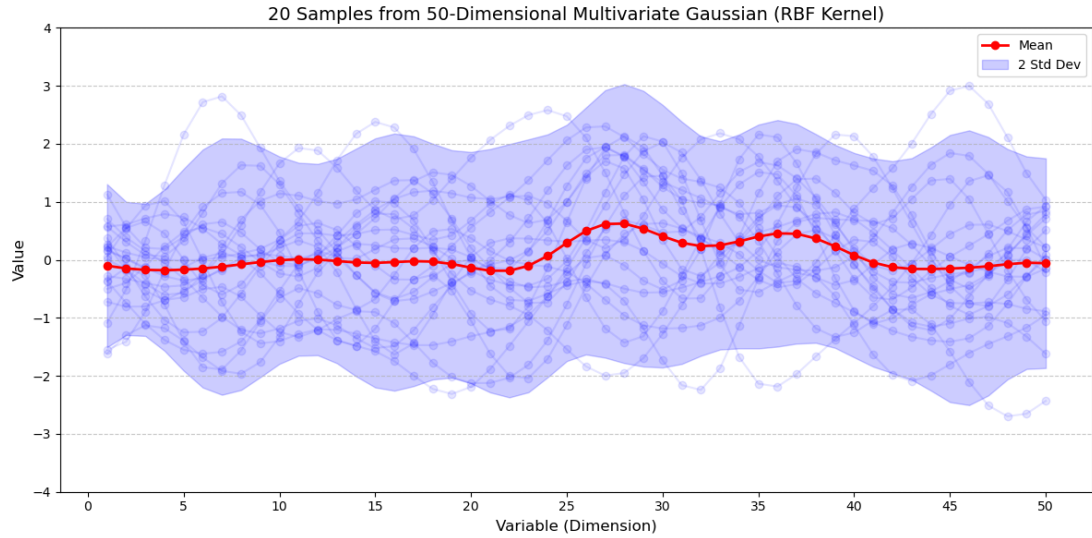


Figure 2.4: 20 samples (blue) from a multivariate Gaussian with  $n = 50$ . Mean and 2 standard deviations are plotted.

Increasing the dimensionality to infinity leads to the concept of a *GP*. We will formally define a GP next, but intuitively, we can think of it as an *infinite distribution*, from which we can *sample functions*. A GP is specified by a *mean function* (in Figure 2.4, this would be the red line), and a *covariance function* which defines how correlated the function values are for any 2 variables. The covariance function would be the RBF kernel function in our example above. The individual samples (connected blue dots) shown in Figure 2.4 represent samples drawn from the GP. Note that the variable dimension should be infinite for our samples, however we can only plot a finite number as an approximation.

One might ask what we gain by doing this. As we will later see, common machine learning tasks like regression can be accomplished using only a single Gaussian! With an intuitive understanding of what a GP is, let us now provide an abstract definition.

**Definition 3. [1]** A GP is an infinite collection of random variables, any finite number of which have a joint Gaussian distribution.

To relate this to what we have explored so far, the *infinite collection of random variables* is the set of  $f(\mathbf{x})$  where  $\mathbf{x}$  ranges over each variable in the input domain, and  $f$  is the sample function we drew. Note that the random variables are not the  $\mathbf{x}$  vectors themselves, but rather the function values evaluated at the  $\mathbf{x}$  vectors. The random variable  $f(\mathbf{x})$  is *indexed* by  $\mathbf{x}$ . In the previous example,  $\mathbf{x}$  was one dimensional (a variable  $x_i$  for some  $i$ ), but we can generalise to a higher number of dimensions, and store  $\mathbf{x} \in \mathbb{R}^n$  where  $n$  is some positive integer.

A GP is completely specified by its mean function and covariance function. The mean function  $m(\mathbf{x})$  and the covariance function (or *kernel*)  $k(\mathbf{x}, \mathbf{x}')$  of a process  $f(\mathbf{x})$  are defined as [1],

$$\begin{aligned} m(\mathbf{x}) &= \mathbb{E}[f(\mathbf{x})], \\ k(\mathbf{x}, \mathbf{x}') &= \mathbb{E}[(f(\mathbf{x}) - m(\mathbf{x}))(f(\mathbf{x}') - m(\mathbf{x}'))]. \end{aligned}$$

Note that these are functions defined on the index of a random variable, rather than the variable itself. The notation,

$$f(\mathbf{x}) \sim \mathcal{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')),$$

denotes the sampling of a function  $f$  from the GP with mean function  $m$  and kernel function  $k$ .

## 2.2 Choice of Kernels

Kernels are an integral part of GPs because they incorporate our prior beliefs of how the function output will behave with respect to its predictors before considering any observed data in the model. The kernel function also determines how *similar* the function outputs should be to other outputs based on how close they are in the input space (e.g. we usually assume some level of smoothness, that is, closer predictor values correspond to closer function output values). We now give the formal definition of a kernel before continuing.

**Definition 4.** [1] A kernel is a function which satisfies the following two properties:

- The kernel function is *symmetric*. That is,  $k(\mathbf{x}, \mathbf{x}') = k(\mathbf{x}', \mathbf{x})$  (the ordering of input elements is irrelevant to the kernel's output).
- Its associated *covariance matrix/gram matrix*,  $K$ , is positive semi-definite (where  $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$  and  $\mathbf{x}_i$  is a real vector).

### 2.2.1 Examples of Covariance Functions

We now cover some important examples of kernels that are commonly used in practice and discuss the meaning of their *hyperparameters* which should be optimised before fitting our model. Before we begin, however, it should be noted that for all of these kernels we could (and likely should) include a random noise hyperparameter,  $\sigma_n^2$ , that is often assumed to be Gaussian to account for random variability in either the data or in the data collection methods.

**The RBF (or Squared Exponential) Covariance Function** [6]: The RBF



covariance function is given by,

$$k_{RBF}(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{|\mathbf{x} - \mathbf{x}'|^2}{\ell^2}\right),$$

where the hyperparameters are  $\ell$  (*characteristic length scale*) and  $\sigma_f^2$  (*signal variance*), and  $|\mathbf{x} - \mathbf{x}'|$  is the euclidean distance between  $\mathbf{x}$  and  $\mathbf{x}'$ . The characteristic length scale determines how much the sample function will vary with its inputs. The signal variance controls the magnitude (or amplitude) of the function. Figure 2.5 visualises the effect that varying  $\ell$  has on random functions drawn from a GP that has an RBF kernel. This covariance function is infinitely differentiable which makes it very smooth and interpretable, which in turn makes it a very common choice to use in practice. However, one must be careful when choosing the RBF kernel as it may enforce smoothness conditions that do not accurately represent the data. In such instances, we could consider using the Matérn covariance function instead (this kernel is defined later).

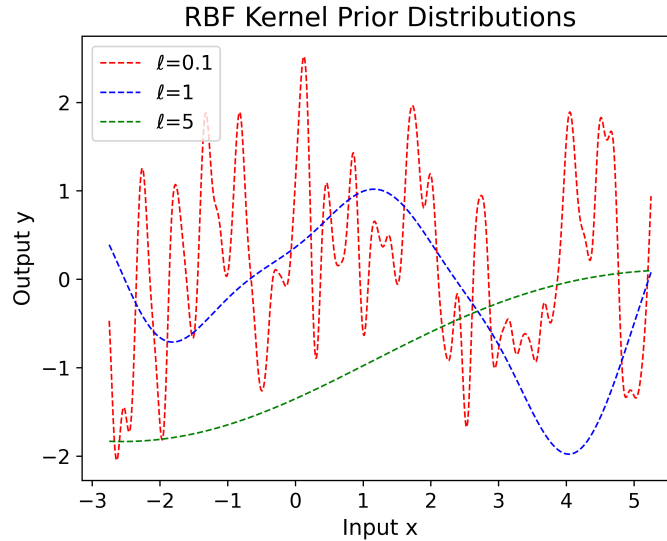


Figure 2.5: GP prior samples for different length scale parameters (here we simply have  $\sigma_f^2 = 1$ ,  $\sigma_n^2 = 0$ ).

**The Polynomial Covariance Function [7]:** Perhaps the most simple class of function, the polynomial covariance function, is given by,

$$k(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^T \mathbf{x}' + \alpha)^\beta,$$

where  $\alpha > 0$  is a *bias* hyperparameter and  $\beta$  is the *degree* of the polynomial. Figure 2.6 gives a visualisation of the kernel's behaviour.

It is also possible to define a more general polynomial covariance function, which uses another kernel  $k_1$  as a base unit (where  $p$  is some polynomial function),

$$k(\mathbf{x}, \mathbf{x}') = p(k_1(\mathbf{x}, \mathbf{x}')).$$

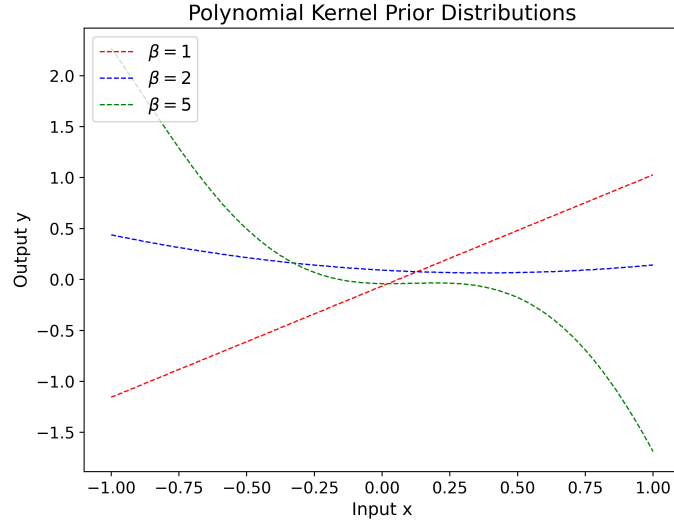


Figure 2.6: GP prior samples for different degrees of polynomials with  $\alpha = \frac{1}{2}$ .

**The Matérn Covariance Function [1]:** The Matérn class of function are another very popular choice of covariance functions. Their functional form is given by,

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \frac{2^{1-\nu}}{\Gamma(\nu)} \left( \frac{\sqrt{2\nu}|\mathbf{x} - \mathbf{x}'|}{\ell} \right)^\nu K_\nu \left( \frac{\sqrt{2\nu}|\mathbf{x} - \mathbf{x}'|}{\ell} \right),$$

where  $\ell$ ,  $\nu$ , and  $\sigma_f^2$  are hyperparameters and  $K_\nu$  is a modified Bessel function [8]. Here  $\ell$  is the characteristic length scale parameter,  $\nu$  is a ‘smoothness’ parameter, and  $\sigma_f^2$  is the signal variance.

When  $\nu = n + \frac{1}{2}$  for  $n \in \mathbb{N}$  the covariance function is given by a polynomial of degree  $n$  and an exponential term. Some of the most common values of  $\nu$  for GPs are  $\nu = \frac{3}{2}$  and  $\nu = \frac{5}{2}$  which are given by,

$$k_{\nu=\frac{3}{2}}(\mathbf{x}, \mathbf{x}') = \left( 1 + \frac{\sqrt{3}|\mathbf{x} - \mathbf{x}'|}{\ell} \right) \exp \left( -\frac{\sqrt{3}|\mathbf{x} - \mathbf{x}'|}{\ell} \right),$$

$$k_{\nu=\frac{5}{2}}(\mathbf{x}, \mathbf{x}') = \left( 1 + \frac{\sqrt{5}|\mathbf{x} - \mathbf{x}'|}{\ell} + \frac{5|\mathbf{x} - \mathbf{x}'|^2}{3\ell^2} \right) \exp \left( -\frac{\sqrt{5}|\mathbf{x} - \mathbf{x}'|}{\ell} \right),$$

respectively. For  $\nu \rightarrow \infty$ , the Matérn kernel is simply the squared exponential kernel. In Figure 2.7 we see the effect changing the smoothness parameter  $\nu$  has on our regression function.

As noted previously, Matérn kernels can often model real data comparatively well compared to the smooth RBF kernel which may impose too strict a restriction on the form of the sampled functions.

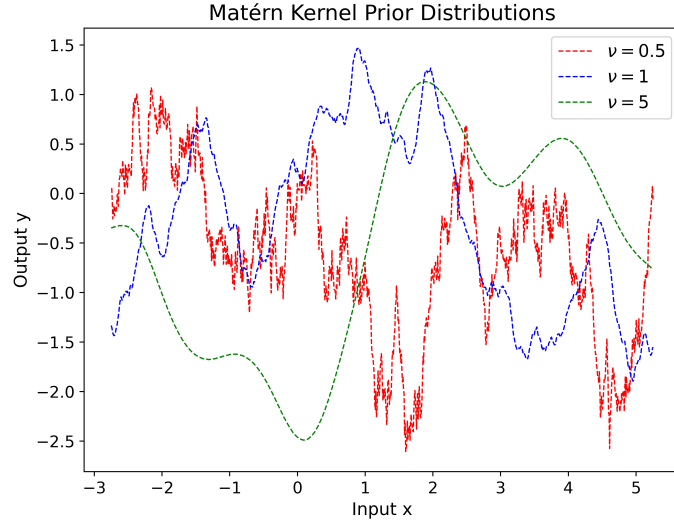


Figure 2.7: Prior samples from the Matérn kernel with different smoothness parameter,  $\nu$ , values (here we simply have  $\ell = 1.0$ ,  $\sigma_f^2 = 1$ , and  $\sigma_n^2 = 0$ ).

**The Periodic Covariance Function [6]:** One final important covariance function we should discuss is the periodic kernel,

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp \left( -\frac{2 \sin^2 \left( \frac{\pi |\mathbf{x} - \mathbf{x}'|}{\theta} \right)}{\ell^2} \right),$$

where  $\ell$  is a length scale parameter,  $\sigma_f^2$  corresponds to the signal variance, and  $\theta$  controls the period. Figure 2.8 shows the results of varying  $\theta$ .

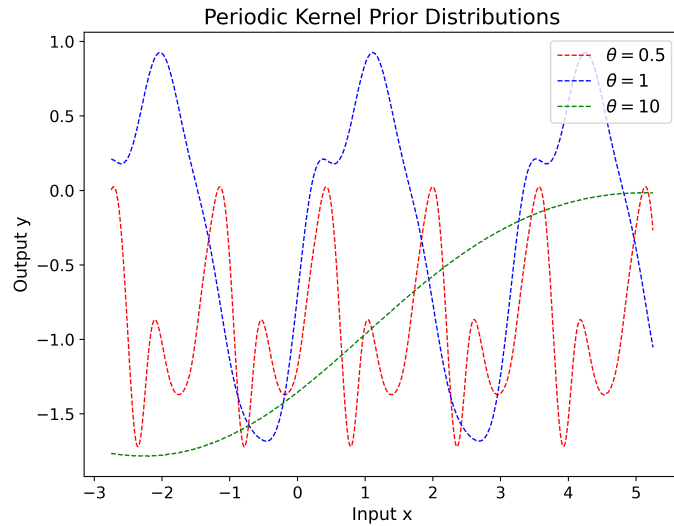


Figure 2.8: Prior samples from the periodic kernel with different period parameter values,  $\theta$  (here we simply have  $\ell = 1.0$ ,  $\sigma_f^2 = 1$ , and  $\sigma_n^2 = 0$ ).

**Input Dimension Dependent Length Scales:** In the above examples, we have solely considered the case where the length scale for each input dimension is the same. This is quite a strong assumption on the data; predictor variables that are uncorrelated with the output of the objective function should have longer length scales than predictors that are highly correlated with the data. We can generalise the kernels given above to accommodate for these potential issues by allowing the length scale to depend on the input dimension. For example, the RBF kernel can be re-defined as [1],

$$k_{RBF}(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp \left( -\frac{1}{2} (\mathbf{x} - \mathbf{x}')^\top L (\mathbf{x} - \mathbf{x}') \right),$$

where  $L = \text{diag}(\boldsymbol{\ell})^{-2}$  and  $\boldsymbol{\ell} = (\ell_1, \ell_2, \dots, \ell_D)$  is a vector containing the length scales corresponding to the  $D$  input dimensions of  $\mathbf{x}$ . This kernel encodes different length scales for each dimension of the input vector  $\mathbf{x}$  and allows the model to be much more flexible over high dimensional input spaces. Constructing kernels in this manner allows the interpretation of the importance of features included in the data through *automatic relevance detection* (ARD). If the length scale of a particular input dimension is very large, this could imply the feature is not necessary for the model, and the GP reflects this with its predictions by effectively ‘ignoring’ changes in this feature. One should be careful however to not drop features without good reason, for example the scale of the feature may simply be very large compared to other features. Chapter 4 of [9] gives further examples of detecting and dealing with irrelevant features using ARD.

### 2.2.2 Constructing New Kernels From Known Kernels

Now that we have seen some common covariance functions we will focus on how to create new kernels from these standard functions. Proofs for these results are given in [10].

**The Sum of Kernels:** If  $k_1(\mathbf{x}, \mathbf{x}')$  and  $k_2(\mathbf{x}, \mathbf{x}')$  are kernels then,

$$k(\mathbf{x}, \mathbf{x}') = k_1(\mathbf{x}, \mathbf{x}') + k_2(\mathbf{x}, \mathbf{x}'),$$

is also a kernel.

**The Product of Two Kernels:** The element-wise product of two kernels is also itself a kernel. If  $k_1(\mathbf{x}, \mathbf{x}')$  and  $k_2(\mathbf{x}, \mathbf{x}')$  are kernels then,

$$k(\mathbf{x}, \mathbf{x}') = k_1(\mathbf{x}, \mathbf{x}') \cdot k_2(\mathbf{x}, \mathbf{x}'),$$

is also a kernel. It is clear to see that these results can be extended by induction to obtain the following valid kernels,

$$\sum_{i=1}^m k_i(\mathbf{x}, \mathbf{x}') \quad \text{and} \quad \prod_{i=1}^m k_i(\mathbf{x}, \mathbf{x}'),$$

assuming  $k_1(\mathbf{x}, \mathbf{x}'), k_2(\mathbf{x}, \mathbf{x}'), \dots, k_m(\mathbf{x}, \mathbf{x}')$  are valid kernel functions.

### 2.2.3 Example of a Custom Kernel

We now show the power of creating and implementing customised kernels to fit the data when we have reason to do so a priori. We consider the air passenger dataset [11] which consists of 144 observations of the number of global air passengers traveling in each month of the year (in hundreds of thousands) from January 1949 up to, and including, December 1960. The data is visualised in Figure 2.9.

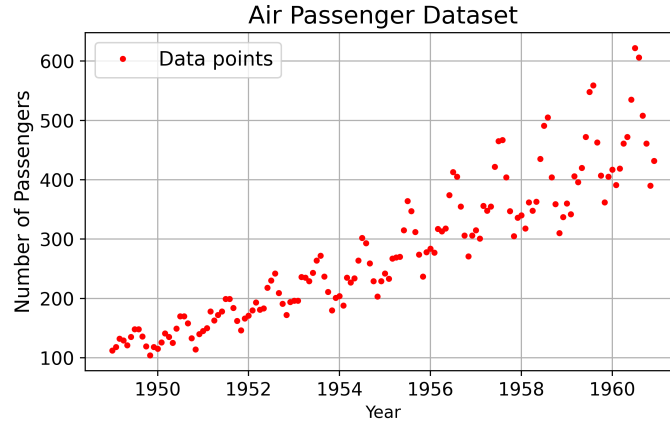


Figure 2.9: Air passenger dataset (the number of passengers is given in hundreds of thousands).

Looking at this data, there appears to be an underlying periodic component that influences the number of monthly passengers. Although the period remains constant, the amplitude increases over time. We model this with a covariance function given by the product of a periodic kernel and a polynomial kernel. The periodic kernel will control the period of the data, while the polynomial kernel will control the dynamically increasing amplitude. This kernel is given by,

$$k_1(x, x') = \theta_1^2 \exp \left( -\frac{\sin^2 \left( -\frac{\pi|x-x'|}{\theta_2} \right)}{\theta_3^2} \right) \cdot \theta_4^2 (x^\top \cdot x' + \theta_5)^{\theta_6},$$

where  $\theta_1, \theta_2, \dots, \theta_6$  are hyperparameters that we will optimise before fitting the model. The interpretations of the hyperparameters are;  $\theta_1$  and  $\theta_4$  are constant amplitude parameters that control the baseline amplitude of the periodic component,  $\theta_2$  controls the period of the periodic component in the data,  $\theta_3$  describes the smoothness of the periodic component,  $\theta_5$  is the bias of the polynomial kernel, and  $\theta_6$  is the degree of the polynomial kernel (the rate of amplitude change).

There also appears to be a rising trend for the data which we would like to model. To do this, we can use a squared exponential kernel with hyperparameters  $\theta_7$

controlling the magnitude of the change and  $\theta_8$  controlling its smoothness,

$$k_2(x, x') = \theta_7^2 \exp\left(-\frac{|x - x'|^2}{\theta_8^2}\right).$$

Notice that this kernel does not actually encode a rising trend in the data before we consider any of the training data. However, this is a desirable property, as it allows the predictions to vary with the data rather than being restricted.

The final component we must add to our covariance function is a noise factor. We define the following kernel to account for the random noise present in the data,

$$k_3(x_i, x_j) = \theta_9^2 \delta_{ij},$$

where  $\theta_9$  controls the magnitude of the random noise component in the data.

With the different components of our custom kernel now defined, we combine them into one custom kernel,

$$k(x, x') = k_1(x, x') + k_2(x, x') + k_3(x, x').$$

To fit these data using the covariance kernel described above, both the predictor and response variables were scaled by subtracting the mean and dividing by the variance of their respective training sets and then these data were translated to have predictor values starting from 0 to ensure that the polynomial kernel modelled the increasing amplitude as intended. The model was then fitted on the scaled and translated training set, and the hyperparameters were optimised by maximising the marginal likelihood using a conjugate gradient-based optimisation scheme. The predictions were then translated and scaled back to the original scale which is shown in Figure (2.10). Table 2.1 gives the performance of our custom kernel against other standard options used for GPs. The Marginal Likelihood metric will be discussed in more detail in Section 6.1.2.

Table 2.1: Marginal likelihoods for our GP model for various kernels.

| Kernel                  | Marginal Likelihood |
|-------------------------|---------------------|
| Custom Kernel           | 54.113              |
| Matérn                  | −17.946             |
| RBF                     | −71.979             |
| Periodic                | −204.328            |
| Polynomial $\times$ RBF | −71.299             |
| Polynomial + RBF        | −71.238             |

When comparing the kernel constructed here with alternative options we see that there is a significant difference in performance when considering a specially constructed kernel based on the data instead of selecting a standard kernel. We also see that the GP gives relatively confident and informative estimates for future observed values, for example, if we consider the estimates given by the Matérn

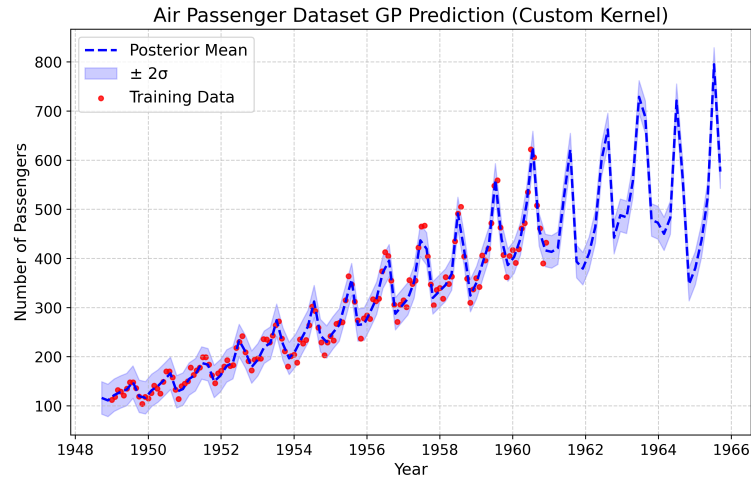


Figure 2.10: GP predictions of the number of global air passengers each month (in hundreds of thousands) using a custom kernel.

kernel in Figure 2.11, we see that one cannot provide any real predictions of how the data will act outside of the training set's range based on these estimates.

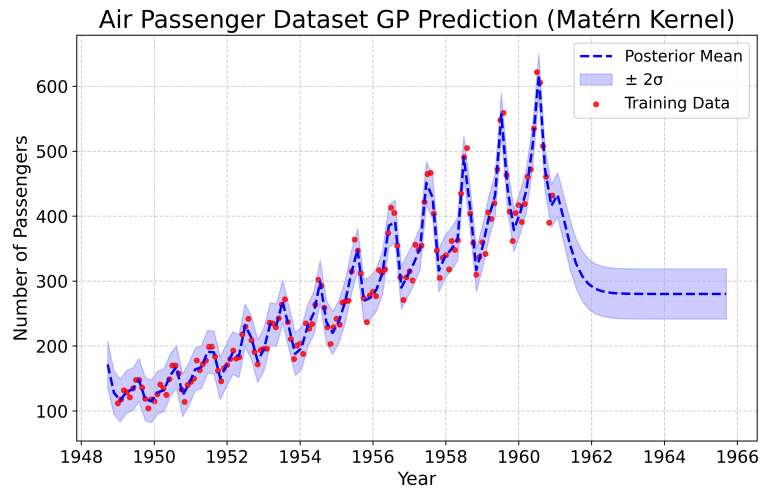


Figure 2.11: GP predictions of the number of global air passengers each month (in hundreds of thousands) using the Matérn kernel.

One should still be wary of using predictions for dates that are far from the training set dates, since there is no data to influence the posterior distribution in these areas. For data such as this, it may be reasonable to assume that the data follows the trend predicted by our model, so our model in this instance could provide us with useful insight into near-future projections of air passenger behaviour. The code for this example can be found in Appendix A.1.

# Chapter 3

## GP Regression

### 3.1 Theory of GP Regression

GPs offer many advantages over traditional methods when applied to regression. Traditional methods typically take a dataset as input and produce a single function that is the best candidate to model the data. However, there might be other candidate functions that provide nearly as good a fit which are simply ignored since only the best function is returned. GPs handle this issue by quantifying the variance in the prediction.

Now motivated by the knowledge that GPs can provide an advantage over traditional methods, let us step through the regression process. This follows [1] closely.

**Step 1:** Place a prior distribution over the functions we believe are possible candidates, given our assumptions of how the data might be distributed. This is specified by the choice of kernel function  $k$ . See Section 2.2 for more details about kernels,

$$f(\mathbf{x}) \sim \mathcal{GP}(\mathbf{0}, k(\mathbf{x}, \mathbf{x}')).$$

**Step 2:** Bring in the dataset  $D = \{(\mathbf{x}_i, f_i = f(\mathbf{x}_i)) : i = 1, \dots, N\}$ . Also define test points  $\{\mathbf{x}_i^*\}_{i=1, \dots, M}$  at which predictions  $f_i^*$  will be made.

**Step 3:** The training and test outputs  $\mathbf{f} = [f_i]_{i=1, \dots, N}$ ,  $\mathbf{f}^* = [f_i^*]_{i=1, \dots, M}$  have the joint distribution ,

$$\begin{bmatrix} \mathbf{f} \\ \mathbf{f}^* \end{bmatrix} | X, X^* \sim \mathcal{N} \left( \mathbf{0}, \begin{bmatrix} K(X, X) & K(X, X^*) \\ K(X^*, X) & K(X^*, X^*) \end{bmatrix} \right),$$



where  $X, X^*$  are the sets of training and test points  $\mathbf{x}_i, \mathbf{x}_i^*$  respectively, and,

$$K(Z, Z') := \begin{bmatrix} k(\mathbf{z}_1, \mathbf{z}'_1) & \cdots & k(\mathbf{z}_1, \mathbf{z}'_k) \\ \cdots & & \cdots \\ k(\mathbf{z}_l, \mathbf{z}'_1) & \cdots & k(\mathbf{z}_l, \mathbf{z}'_k) \end{bmatrix},$$

for some  $Z = \{\mathbf{z}_1, \dots, \mathbf{z}_l\}$ ,  $Z' = \{\mathbf{z}'_1, \dots, \mathbf{z}'_k\}$ .

To make a prediction for  $\mathbf{f}^*$ , condition the joint distribution on the observations,

$$\begin{aligned} \mathbf{f}^* | X^*, X, \mathbf{f} &\sim \mathcal{N}(K(X^*, X) \cdot K(X, X)^{-1} \cdot \mathbf{f}, \\ &K(X^*, X^*) - K(X^*, X) \cdot K(X, X)^{-1} \cdot K(X, X^*)). \end{aligned}$$

**Step 4:** The kernel hyperparameters are optimised using standard optimisation methods.

**Remarks:** In Step 1, a mean  $\mathbf{0}$  is assumed, however this need not be the case. This is often assumed when we have little knowledge of the data, and data can always be normalised to have mean  $\mathbf{0}$ .

A full derivation of the final equation from Step 3 is given in [1]. This expression can cause numerical issues in implementations due to the inversion of the matrix  $K(X, X)$ . Methods to deal with this include solving the corresponding linear system, or using decompositions of the matrix, such as the Cholesky decomposition [12]. For example, to calculate the mean of the distribution, we could instead solve the linear system  $K(X^*, X) \cdot \boldsymbol{\alpha} = \mathbf{f}$  for  $\boldsymbol{\alpha}$  using Cholesky, (or even Gaussian elimination), and then calculate the mean as  $K(X^*, X) \cdot \boldsymbol{\alpha}$  (substitute  $\mathbf{f}$  for our new expression).

With the conditional distribution in Step 3, the best-estimate mean function and variance in predictions have been quantified. Figure 3.1 illustrates this idea by providing 1 training point (left) and 5 training points (right). The red line indicates the mean prediction from 50 test points, and the shaded blue region gives 1 standard deviation.

The kernel function  $k$  is often parametrised. For example, the RBF kernel (Section 2.2.1) is defined by the characteristic length scale and the signal variance. In Step 4 the hyperparameters are optimised to ensure the best fit to our data. This concept is discussed in great detail in chapter 6, where we explore some novel methods of doing this. Existing methods include cross-validation, or maximising the marginal likelihood.

The expression for log marginal likelihood is defined in Section 6.1.2, given the hyperparameters  $\boldsymbol{\theta}$  specific to the choice of kernel. This is the more general expression, where noise is modelled through  $\mathbf{y} := (y_i)_i$  with  $y_i := f(\mathbf{x}_i) + \epsilon_i$  where  $\epsilon_i$  is generally assumed to be Gaussian, and small diagonal terms are added to

$K$ , the covariance matrix (which depends on  $\theta$ ). This helps to resolve numerical issues. Standard optimisation techniques, such as gradient descent, can then be used to maximise this quantity while adjusting the hyperparameters  $\theta$  of the kernel function.

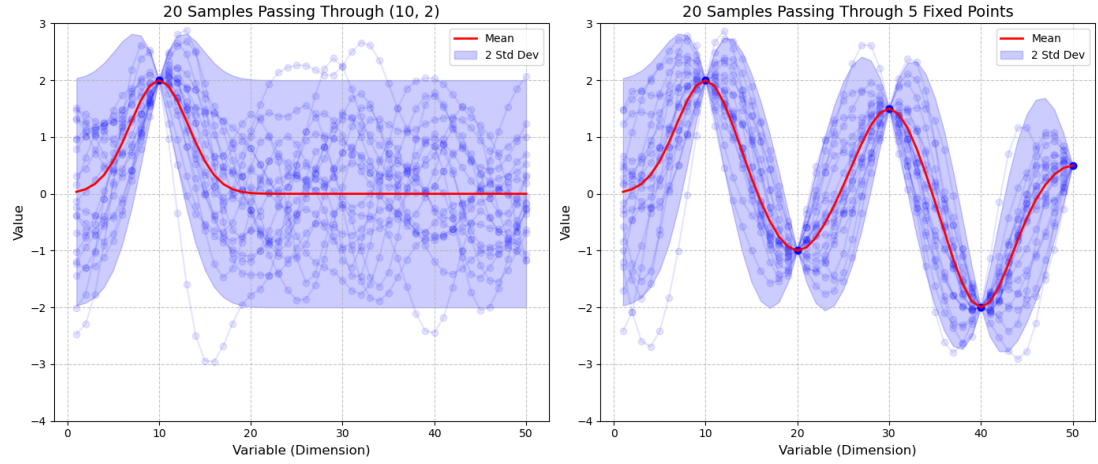


Figure 3.1: 20 samples (blue) from a multivariate Gaussian with 50 dimensions. Mean and 2 standard deviations are plotted. 1 point (*left*) and 5 points (*right*) have been fixed, and samples updated according to the posterior distribution.

## 3.2 Example: Snowline Dataset Regression

### 3.2.1 Dataset & Kernel Choice

In this section, we explore how GPs can be used for regression on a real dataset. We will explore historical records of the snowline in the Scottish mountains, which is the minimum altitude at which snow is present at a given location and time. The data comes from two sources; the Snow Survey of Great Britain provides daily observations of the snowline by human observers at selected locations [13], and the Met Office HadUKGrid datasets [hadukgrid] provide high resolution meteorological data for a variety of timescales, up to daily.

Figure 3.2 shows the snowline data on 31 December 1970 at the observed locations on that day. The data is split into a simple test-train partition to quantify how well the GP performs. It is normalised (to mean 0) to help ensure we obtain well-conditioned covariance matrices since matrix inversion is crucial to GP regression. A gradient-based optimisation algorithm is used to find the best hyperparameters for the kernel. The Matérn kernel is chosen for this task, which is a middle ground between smooth and noisy kernels. The dataset contains noise by nature, consisting of human observations, so it makes more sense to use the Matérn kernel than a smooth kernel like the RBF kernel.

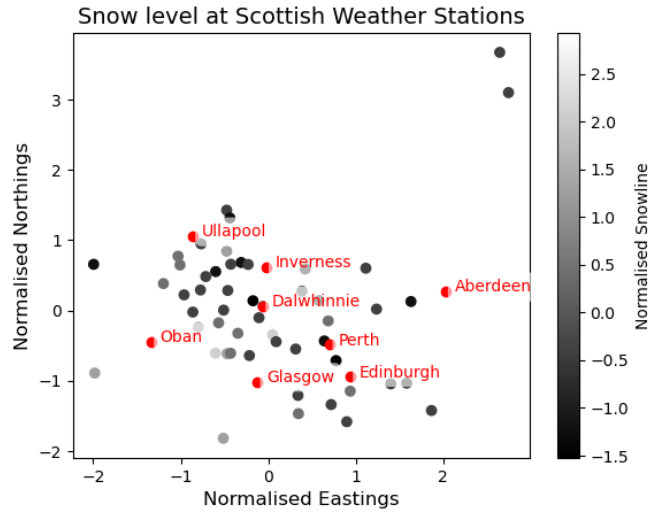


Figure 3.2: Normalised snowline at select locations across Scotland on 31 December 1970.

### 3.2.2 GP Predictions

Figure 3.3 shows the mean function and variance predictions from the trained GP. At an initial glance, the mean function appears to model the data quite well where the data is more dense. Empirically speaking, we are able to separate low-snow-level areas from high-snow-level areas, and two clusters of low-snow-level are identified.

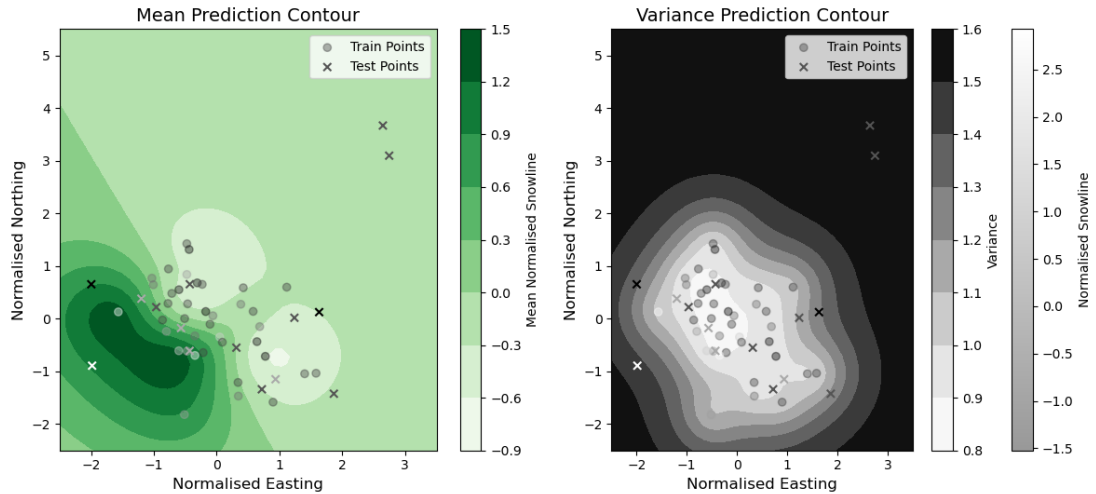


Figure 3.3: Contour plots for mean function and variance from GP predictions, dataset 31 December 1970. Training and test points are shaded according to true snowline.

Away from the dense data, there is not much information from which the posterior distribution is affected, so, as expected, the mean function levels out to zero. The variance plot confirms this as expected; variance in the predictions is lowest where the majority of data can be found and highest where data is sparse.

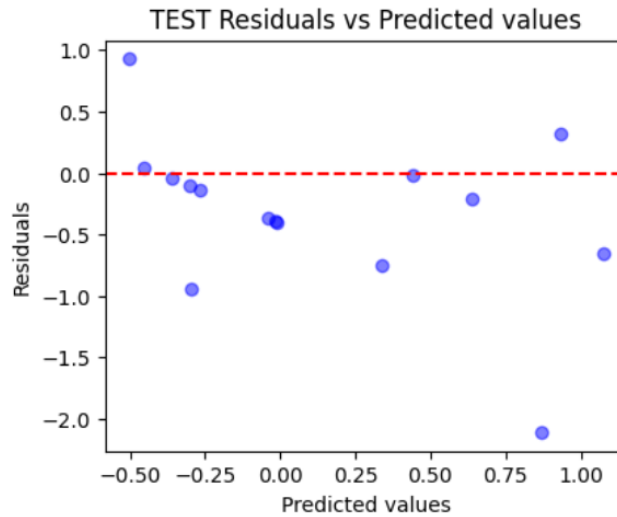


Figure 3.4: Residual plot (*actual value* – *mean prediction*) for test set.

Figure 3.4 gives the residual plot for prediction errors. With the exception of one point, all errors are within 1 standard deviation of the correct prediction. The errors are evenly distributed on the whole, however, there appears to be slight unwanted linear correlations. This could, however, be a consequence of the sampling method of the data. Observations were made daily by human observers and snowline was recorded in (approximate) 150m bands rather than the exact true elevation.

### 3.2.3 Limitations

It is difficult to make predictions where there is no data while using GPs. It is therefore not possible using the model above to predict snowline in England or Wales, given that the data we have is only for Scottish regions. An attempt to fix this issue is to re-frame the problem so that we no longer search for functions in a coordinate-based input space. Instead, we could use other data such as rainfall or temperature to make predictions. Other geographical areas like England and Wales are then more applicable since, for example, the temperature data will be in a comparable range to Scotland.

Other general limitations to this example include the fact that the posterior distribution must be re-calculated whenever we change the day from which we use the data, which is an expensive operation. Given that Scottish weather is highly variable, data from two given days could be significantly different, resulting in incorrect predictions unless we recompute the posterior.

### 3.2.4 An Alternative Approach and Comparison to a Meteorological Model

In the second part of this example, we explore the HadUKGrid datasets from the Met Office and CEDA [hadukgrid]. We again use the snowline dataset from before. Looking at Figure 3.5, there is unfortunately not much of an immediate correlation between rainfall, temperature and snowline. We could perhaps say that for locations with more rainfall (which includes snow) there is a lower snowline (as would be expected).

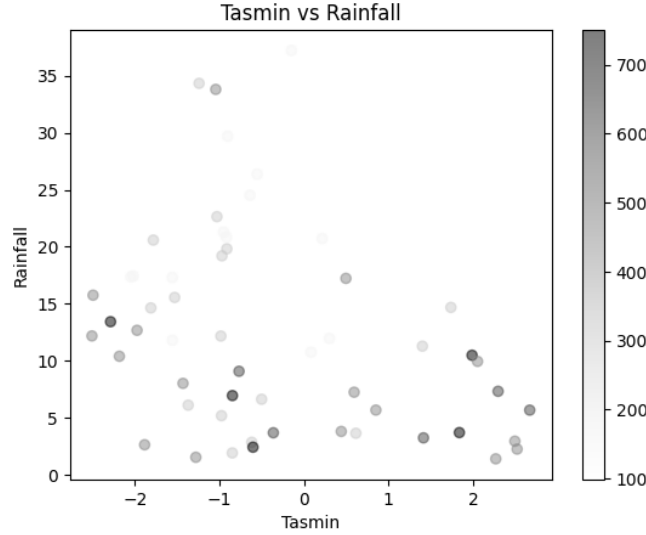


Figure 3.5: Plot of minimum temperature (degrees C) vs. rainfall (mm). Snowline is represented using shading as before.

Given the visibly low correlations in the data, we must expect that performing GP regression will not yield strong results. However, it will still be beneficial to compare with the predictions from a meteorological model to gauge how well a GP performs.

We can define the formula for the vertical temperature gradient [14] (i.e. rate of change of temperature with height) as follows,

$$-\frac{dT}{dz} = \frac{g}{c_p} = \Gamma_d,$$

where  $T$  is temperature,  $z$  is height,  $g \approx 9.81$  is the gravitational constant,  $c_p$  is specific heat capacity, and  $\Gamma_d$  is the tropospheric standard lapse rate. A typical  $\Gamma_d$  is between 6 and 7 degrees celsius per km [15], so we will assume  $\Gamma_d = 6.5$  henceforth.

Precipitation falls as snow when the air temperature is below 2 degrees celsius [16]. Hence, given the elevation, precipitation and temperature at a weather

station, we can use the lapse rate to get an estimate for snow line, i.e., we solve,

$$2 = T_{station} - 6.5(z - z_{station}),$$

for  $z$ , the height at which precipitation falls as snow. We also need to consider whether precipitation occurred at the time of interest.

Figure 3.6 gives the snowline predictions from both the GP and the Meteorological model. The GP was trained using the product of Matérn and Rational Quadratic kernels (various kernels were tested, and this choice yielded the best results; Section 3.3 gives details on kernel selection for the original dataset in the previous section). Both predictions are quite noisy due to the nature of the dataset, however the GP regression resulted in a Root Mean Square Error (RMSE) of 129.634 compared to 248.236 for the meteorological model. Thus, using GPs provides a benefit over the meteorological model used to predict snowline. Considering snowline was recorded to the nearest 150m in the dataset, the RMSE for GPs is within one 150m “band” of error, whereas it is nearer two bands for the meteorological model.

It would be interesting to examine other datasets to see if a GP could produce a better prediction. The issue with the Scottish snowfall data is that it is quite limited in scope. More detailed data could, perhaps, be generated using historical satellite imagery. Automated recording methods could be more accurate and provide more precise data upon which to train our models.

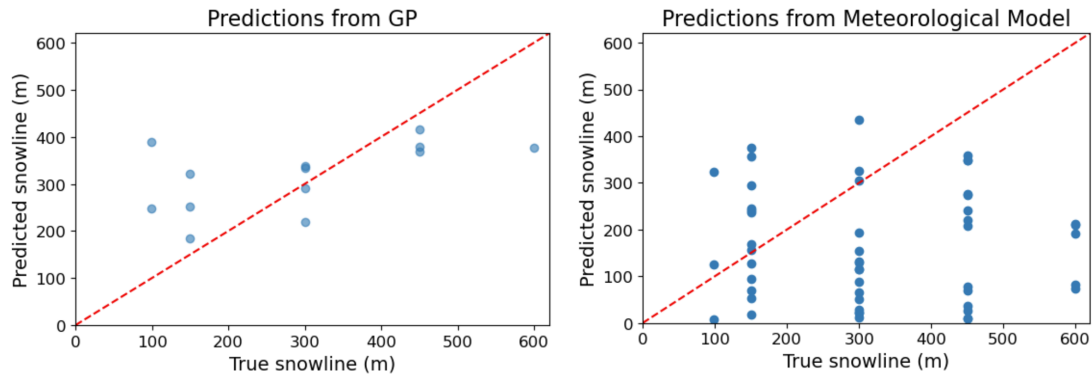


Figure 3.6: *(left)* Predicted snowline against True snowline from the GP using the product of Matérn and Rational Quadratic Kernels. *(right)* Predicted snowline against True snowline from the meteorological model. (Note, a test-train split was not required for the meteorological model since it does not need trained, hence there are more predictions on the graph).

### 3.3 Impact of Kernel Choice

We now examine the impact the choice of kernel has on the performance of a GP model on the snowline dataset from Section 3.2. GP regression was performed using various kernel types, including Matérn, RBF, and polynomial kernels, and

sums and products of these. Figure 3.7 gives the best and worst predictions.

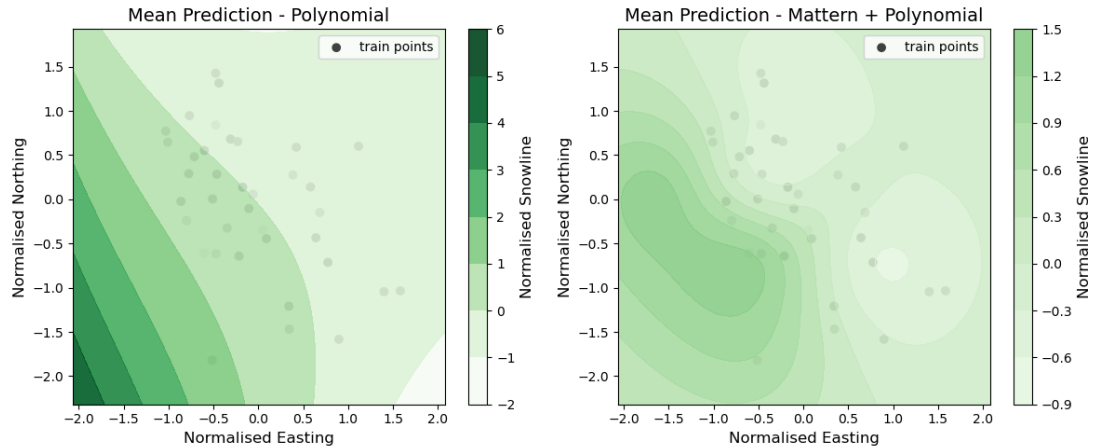


Figure 3.7: Best (*right*) and worst (*left*) performing kernel in GP regression on snowline dataset.

The polynomial kernel is the worst-performing kernel because it does not have the flexibility required to model the complexity of the dataset. Table 3.1 quantifies the performance of each model by listing the RMSE for each of the kernel types which were tested. We confirm our hypothesis in Section 3.2 that the Matérn kernel outperforms the RBF kernel. The best performing model is the sum of the Matérn and polynomial kernels but the advantage over the Matérn kernel is negligible; the polynomial kernel does not add any meaningful modelling power to what the Matérn kernel offers.

Table 3.1: RMSE for GP predictions on test set for various kernels.

| Kernel                     | RMSE     |
|----------------------------|----------|
| Matérn + Polynomial        | 0.721 75 |
| Matérn                     | 0.721 76 |
| Matérn + RBF               | 0.721 90 |
| RBF                        | 0.755 35 |
| Matérn $\times$ Polynomial | 0.757 44 |
| Polynomial                 | 0.974 15 |

The code for this chapter is included Appendix A.2 for reference.

# Chapter 4

## GP Classification

### 4.1 Theory of GP Classification

GPs also offer advantages over traditional methods when applied to classification. Traditional classification methods typically take a dataset as input and produce linear decision boundaries that assign each input to a specific class. GPs quantify the uncertainty in the class predictions and provide a probabilistic output for a given input. In particular, where the given input is far from any training data, the GP prediction will revert to the prior and return a maximally uncertain prediction. GPs also do not make the assumption that the decision boundary is linear and can incorporate information about the underlying decision boundaries through the kernel function.

Figure 4.1 compares the traditional logistic regression approach with the GP classification approach. Notice that while Logistic Regression produces a linear decision boundary, the GP classifier produced a curved boundary that better captures the underlying class distribution. In regions lacking training data, the GP classifier is maximally uncertain, assigning equal probabilities to both classes,  $p(\text{class} = -1) = p(\text{class} = 1) = 0.5$ .

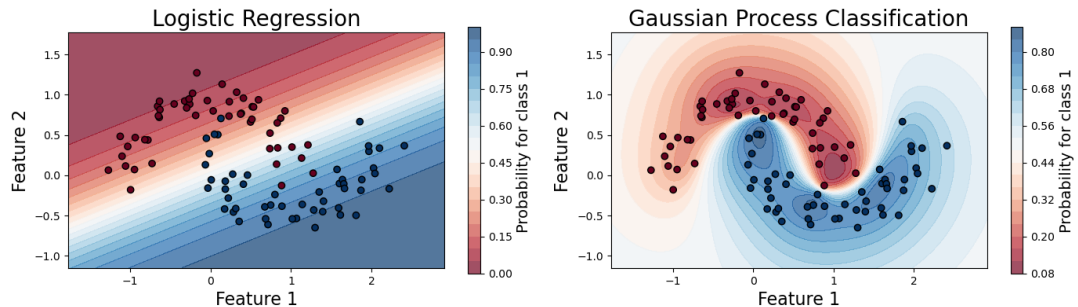


Figure 4.1: Comparison of probability maps for Logistic Regression and GP classification.



In binary classification, the class of a data point  $\mathbf{x}_i$  is denoted by  $y_i \in \{+1, -1\}$ , and the training dataset can be written  $\{(\mathbf{x}_i, y_i)_i\}$ . Let  $X$  and  $\mathbf{y}$  denote the sets of  $\mathbf{x}_i$  and  $y_i$  respectively. Suppose that we also have a set of test points  $\mathbf{x}^* = \{(\mathbf{x}_j^*)_j\}$  for which we want to predict the class labels. Our goal is to model a function  $\pi$  from the domain of the  $\mathbf{x}$  points (e.g.  $\mathbb{R}^{\dim \mathbf{x}}$ ) to the interval  $[0, 1]$ , so that  $\pi(\mathbf{x}) = p(y = +1|\mathbf{x})$  and  $1 - \pi(\mathbf{x}) = p(y = -1|\mathbf{x})$  for some arbitrary point  $(\mathbf{x}, y)$ .

To do this, we would like to place a probability distribution over this unknown function  $\pi$ . This is similar to the regression section, but here the label  $y$  is discrete  $y \in \{-1, +1\}$  whereas in regression  $y \in \mathbb{R}$ . Notice also that we do not get to see the true probability distribution  $\pi(\mathbf{x})$  directly from our training data, but rather a sample from the corresponding distribution  $p(y|\mathbf{x}) = \pi(\mathbf{x})$  through the class label  $y$ . We cannot simply assume  $\pi(\mathbf{x})$  is distributed according to some Gaussian since the probability mass would not sum to 1 (the tails of the distribution will always lie outwith the range  $[0, 1]$ ).

Instead, we first draw a *latent function*  $f(\mathbf{x})$  from a GP prior  $f(\mathbf{x}) \sim \mathcal{GP}(\mathbf{0}, k(\mathbf{x}, \mathbf{x}'))$ , and then map the function value into the range  $[0, 1]$  which can be accomplished using the sigmoid function [17]. The probability  $\pi(\mathbf{x})$  can then be modelled with,

$$\pi(\mathbf{x}) = \sigma(f(\mathbf{x})) = \frac{1}{1 + \exp(-f(\mathbf{x}))},$$

and we use the fact  $f$  was drawn from a GP to quantify a distribution over  $\pi$ . Figure 4.2 illustrates the “squishing” process by applying the sigmoid function to a sample  $f$  from the GP prior.

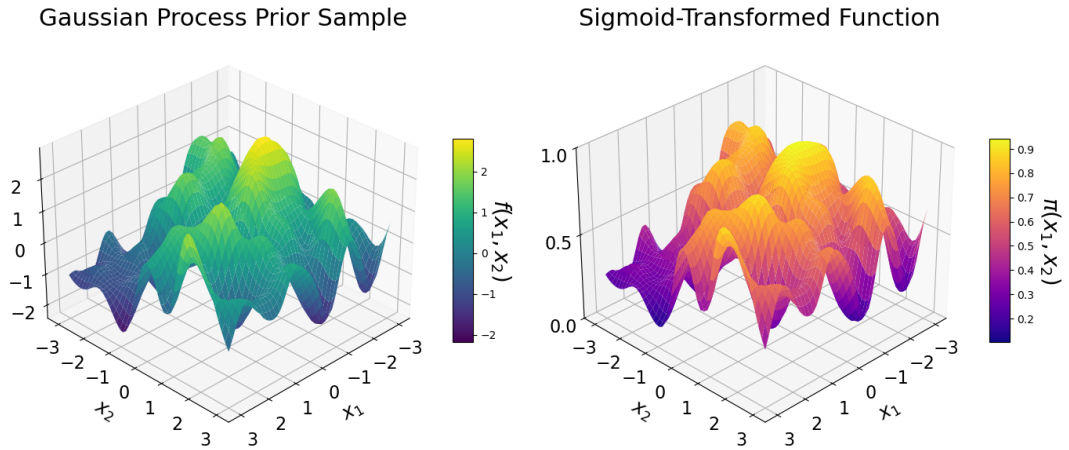


Figure 4.2: One sample drawn from the GP prior on 2 dimensions, then scaled to the range  $[0, 1]$  using the sigmoid function.

Predictions  $\pi_i^*$  for the class label of test points  $\mathbf{x}_i^*$  can then be made in two steps [1]. Let  $\mathbf{f} := (f_1 := f(\mathbf{x}_1), \dots, f_N := f(\mathbf{x}_N))^T$  denote the latent function values at training points.

**Step 1: Compute the posterior distribution of the latent function value at test points,**

$$p(f_i^*|X, \mathbf{y}, \mathbf{x}_i^*) = \int p(f_i^*|X, \mathbf{x}_i^*, \mathbf{f})p(\mathbf{f}|X, \mathbf{y})d\mathbf{f}.$$

**Step 2: Produce the predicted class probabilistic distribution,**

$$\pi_i^* = p(y_i^* = +1|X, \mathbf{y}, \mathbf{x}_i^*) = \int \sigma(f_i^*) p(f_i^*|X, \mathbf{y}, \mathbf{x}_i^*) df_i^*.$$

The posterior distribution  $p(\mathbf{f}|X, \mathbf{y})$  over the latent function values  $\mathbf{f}$  can be computed using Bayes' rule,

$$p(\mathbf{f}|X, \mathbf{y}) = \frac{p(\mathbf{y}|\mathbf{f})p(\mathbf{f}|X)}{p(\mathbf{y}|X)}.$$

The likelihood  $p(\mathbf{y}|\mathbf{f})$  is given by the product  $\prod_i p(y_i|f_i)$  with  $p(y_i = +1|f_i) = \sigma(f_i)$  and  $p(y_i = -1|f_i) = 1 - \sigma(f_i)$ . The prior  $p(\mathbf{f}|X)$  is normally distributed according to the GP ( $\sim \mathcal{N}(\mathbf{0}, K)$ ). The evidence  $p(\mathbf{y}|X) = \int p(\mathbf{y}|\mathbf{f})p(\mathbf{f}|X)d\mathbf{f}$  causes issues, since the integral is analytically intractable [1].

The Laplace approximation is a method of approximating a distribution over continuous variables with a Gaussian. We will use it to approximate the intractable posterior  $p(\mathbf{f}|X, \mathbf{y})$ . [1] gives an overview which we will explain, and we will additionally derive some of the statements here. Informally, Laplace approximation finds the Maximum a Posteriori (MAP) point estimate of the posterior distribution  $p(\mathbf{f}|X, \mathbf{y})$  and then uses a Taylor expansion around this point to estimate the geometry of the distribution [18].

The first step is to find the MAP estimate, or *mode*  $\hat{\mathbf{f}}$ , by maximising the log posterior,  $\log p(\mathbf{f}|X, \mathbf{y})$ , where  $\log p(\mathbf{f}|X, \mathbf{y}) \propto \log p(\mathbf{y}|\mathbf{f}) + \log p(\mathbf{f}|X)$ ,

$$\hat{\mathbf{f}} := \arg \max_{\mathbf{f}} [\log p(\mathbf{y}|\mathbf{f}) + \log p(\mathbf{f}|X)].$$

This can be accomplished using common gradient-based methods, for example, Newton–Raphson iterations [19]. This is an iterative approach which repeatedly computes a new guess  $\mathbf{f}^{new}$  for the mode  $\hat{\mathbf{f}}$ ,

$$\mathbf{f}^{new} = \mathbf{f} - (\nabla^2 \Psi(\mathbf{f}))^{-1} \nabla \Psi(\mathbf{f}),$$

where  $\Psi(\mathbf{f})$  denotes the log-posterior. Figure 4.3 provides an illustration of the convergence procedure: the algorithm iteratively refines the estimate of  $\hat{\mathbf{f}}$  by combining prior beliefs encoded in  $\log p(\mathbf{f}|X)$  (which regularises the solution) with the likelihood term  $\log p(\mathbf{y}|\mathbf{f})$  (which ensures consistency with the observed data). Each Newton-Raphson update adjusts  $\mathbf{f}$  to better combine these two sources of

information, gradually shifting toward regions of higher posterior density.

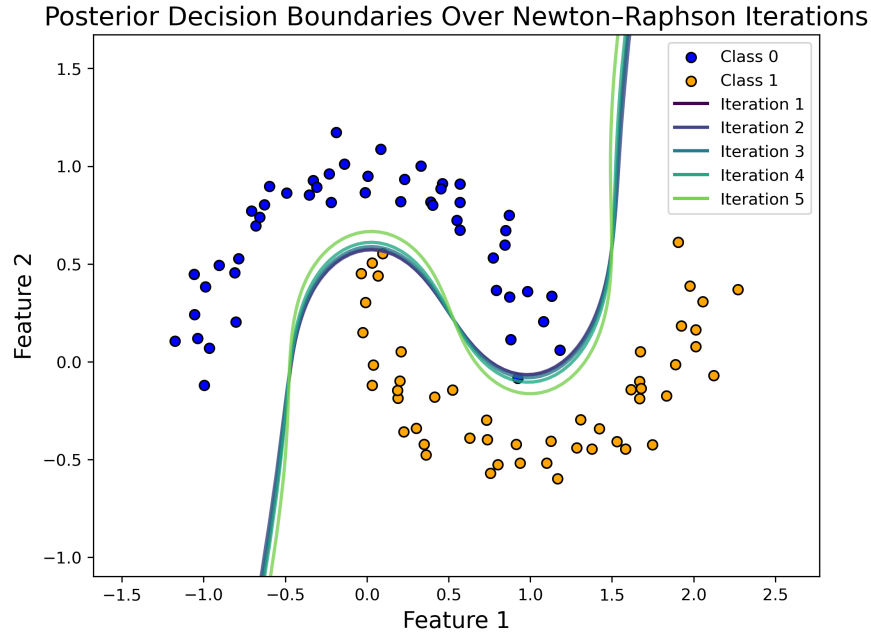


Figure 4.3: Progression of the decision boundary contour during Newton-Raphson optimisation iterations, illustrating convergence to the mode  $\hat{\mathbf{f}}$  by maximising the log-posterior.

Now equipped with the MAP point estimate, the next step is to compute the second-order Taylor expansion of the (log) posterior around  $\hat{\mathbf{f}}$ . Since  $\hat{\mathbf{f}}$  is the maximiser of the log posterior, the first derivative at  $\hat{\mathbf{f}}$  will be zero. This leaves,

$$\begin{aligned} \log p(\mathbf{f}|X, \mathbf{y}) &\approx \log p(\hat{\mathbf{f}}|X, \mathbf{y}) + \\ &\quad \frac{1}{2}(\mathbf{f} - \hat{\mathbf{f}})^T [\nabla_{\mathbf{f}}^2 \log p(\hat{\mathbf{f}}|X, \mathbf{y})](\mathbf{f} - \hat{\mathbf{f}}) + \\ &\quad \mathcal{O}((\mathbf{f} - \hat{\mathbf{f}})^3), \end{aligned}$$

where  $\mathcal{O}$  is the usual asymptotic notation:  $g(x) = \mathcal{O}(f(x)) \iff \frac{f(x)}{g(x)}$  remains bounded as  $x \rightarrow \infty$ .

Exponentiating the above, this then yields  $p(\mathbf{f}|X, \mathbf{y}) \propto \exp(\frac{1}{2}(\mathbf{f} - \hat{\mathbf{f}})^T A(\mathbf{f} - \hat{\mathbf{f}}))$  with  $A = \nabla_{\mathbf{f}}^2 \log p(\hat{\mathbf{f}}|X, \mathbf{y})$ . Comparing this expression to a standard Gaussian (defined in Section 2.1), it becomes apparent that it is reasonable to approximate the posterior with a Gaussian. Laplace approximation approximates the posterior  $p(\mathbf{f}|X, \mathbf{y})$  using the following Gaussian:

$$q(\mathbf{f}|X, \mathbf{y}) := \mathcal{N}(\mathbf{f}|\hat{\mathbf{f}}, -A).$$

Then for a test input  $\mathbf{x}^*$  the predictive mean and variance can be computed using standard techniques for Gaussians (as in the regression section, [1] gives the exact formulas) to obtain the Gaussian conditional distribution  $f^*|X, \mathbf{y}, \mathbf{x}^*$ .

Finally, the class probability for  $y^* = +1$  can then be obtained through,

$$\pi(\mathbf{x}^*) = q(y^* = +1|X, \mathbf{y}, \mathbf{x}^*) = \int \sigma(f^*) q(f^*|X, \mathbf{y}, \mathbf{x}^*) df^*.$$

The Laplace approximation often fails in high dimensions where the posterior has non-Gaussian features (e.g., asymmetry, heavy tails) that aren't captured by matching just the mode and curvature. This occurs because the approximation ignores higher-order moments and dependencies, which dominate in sparse data regimes [20].

## 4.2 Application to Heart Disease Dataset

### 4.2.1 Data Analysis

This section will explore a real-world binary classification application using a heart disease dataset. This data set dates from 1988 and consists of four subsets: Cleveland, Hungary, Switzerland, and Long Beach V [21]. It contains features such as age, sex, chest pain type, resting blood pressure, maximum heart rate achieved (thalach), and various other medical indicators relevant to heart disease. We also have a 'target' value to indicate whether or not a person has a heart disease (1 represents having a disease and 0 represents not having a disease). We use these features to classify patients into two categories: +1 for 'with heart disease' and -1 for 'without heart disease'. For the purposes of this experiment, we only consider a subset of the data that consists of every third entry of both the training and test set.

In this experiment, we select 'age' and 'thalach' as the two features used to construct a GP classification model, as these two features are biologically significant to heart disease [22]. We select only two predictors as we aim to find a simple model to ensure the results are interpretable and explainable while also ensuring the Laplace approximation is valid (i.e. we do not have a large input dimension).

The data is normalised to  $[0, 1]$ , ensuring both features have the same scale before fitting the model (this reduces numerical issues). The rational quadratic kernel [1] is used for the latent function, yielding the best results. It can be seen as the infinite sum of squared-exponential kernels with different length scales and is defined as,

$$k(\mathbf{x}, \mathbf{x}') = \left(1 + \frac{|\mathbf{x} - \mathbf{x}'|^2}{2\alpha\ell^2}\right)^{-\alpha},$$

where  $\alpha$  and  $\ell$  are hyperparameters.

We start by creating a logistic regression baseline model. The logistic regression model is trained and its performance is evaluated using a test set. The results are summarised in Table 4.1. Figure 4.4 shows the baseline model's predictive

probabilities over the input space.

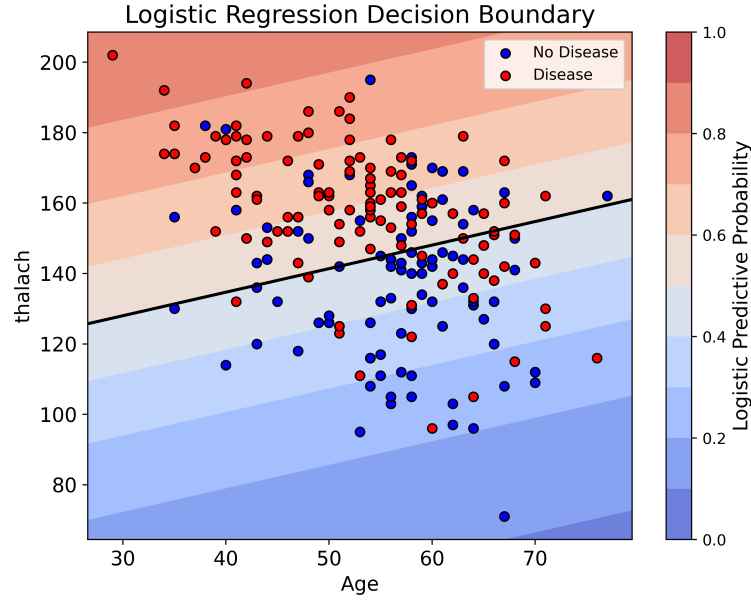


Figure 4.4: The predictive probabilities of the baseline model (logistic regression) and the decision boundary.

### 4.2.2 Applying the Laplace Approximation

We then fit the proposed GP classification model using the Laplace approximation. Figure 4.5 shows the predictive probabilities of the model and its decision boundary.

The accuracy, precision, recall and F1 score of each model are presented in Table 4.1. A higher accuracy means that the method correctly classifies more data points, while a higher precision means that there are less false positives. The recall value is the proportion of true positive results correctly identified by the model, and the F1 score is a metric that combines the accuracy and the precision of the model. A higher F1 score implies a better balance between the precision and recall of the model [23].

From Table 4.1, we see the baseline model significantly underperforms when compared to the GP classification model. Looking at Figure 4.4 and Figure 4.5 we can see that the GP model captures significantly more of the variability within the data, which is not surprising because looking at the data we can easily observe that it is not linearly separable. The logistic model is restricted to a linear prediction for the boundary line, however the GP model is able to capture the non-linear behaviour much more accurately by shaping its predictions around the data. Looking at all of the metrics provided, we see that the GP model provides better estimates for the data in terms of fewer false positives and a higher value of true positives classified as well.

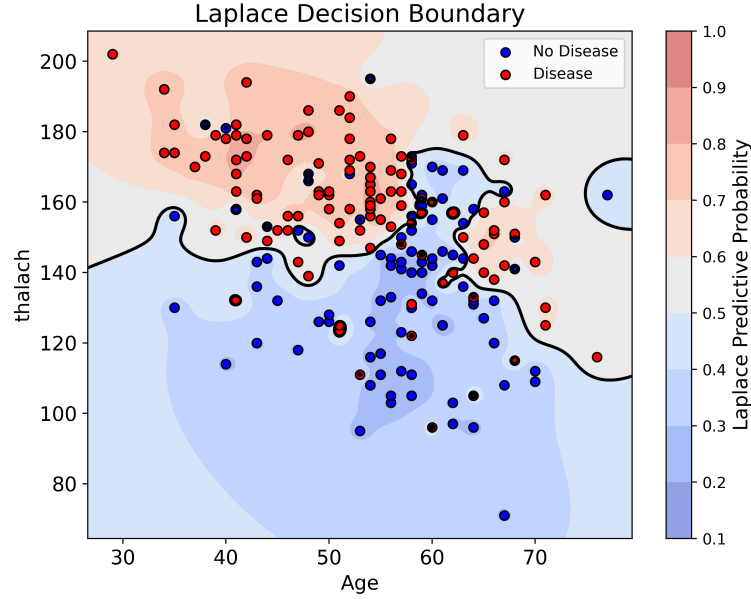


Figure 4.5: Predictive probabilities and decision boundary for the Laplace approximation against age and thalach. The black line is the boundary along which the probability of the class being +1 and -1 is 0.5.

We must also consider the impact of over-fitting on the inference drawn from Figure 4.5. The model is fit very closely to the training data in regions where there are apparent outliers to the overall trend, however, since the predictive accuracy is still very high for the test set, it is reasonable to assume that the model is not significantly over-fitted to the whole dataset. Therefore, we might only consider questioning the predictions in some very small regions since the general probability distribution appears to fit the data well.

The improvement over the baseline model in this simple example illustrates the power of GPs in a classification setting to explain non-linear trends in classification approximations to achieve significant improvement over standard methods for binary classification.

Table 4.1: Performance comparison between Laplace and EP.

| Method   | Accuracy | Precision | Recall | F1 Score |
|----------|----------|-----------|--------|----------|
| Baseline | 0.594    | 0.564     | 0.667  | 0.611    |
| Laplace  | 0.812    | 0.794     | 0.818  | 0.806    |

The code for this chapter is included in Appendix A.3 for reference.

# Chapter 5

## Training Point Selection for GPs

The behaviour of GP models is especially sensitive to the selection of training points [24]. In practice, the accuracy, efficiency, and generalisation of a GP depend not only on the quantity of data, but are also significantly reliant on the locations where data are sampled. Poorly chosen training points can lead to overfitting in densely sampled regions or significant uncertainty in areas with sparse data.

This chapter examines training point selection in two major approaches within the GP framework: model-free and model-based sequential designs. The former can provide an initial sampling strategy for the latter. Moreover, understanding sequential design helps to understand the iterative nature of surrogate-assisted Bayesian optimisation in Chapter 6. In combination, these strategies form a robust training point selection framework by combining the benefits of an evenly spaced starting design with the concentrated refinement offered by sequential designs to reduce uncertainty.

### 5.1 One-shot Training Point Selection

This section examines the model-free one-shot training point selection for GP models, which are one-shot approaches that select training points before any model fitting occurs. We begin by briefly introducing a simple random initialisation using Monte Carlo sampling. Although easy to implement, Monte Carlo sampling can often lead to uneven distributions and clustering in the input space [25].

In contrast, space-filling designs offer a more uniformly distributed set of training points. Accordingly, we focus on two widely used space-filling techniques: Latin Hypercube Sampling and Sobol Sequences. The performance of these methods will be compared with that of Monte Carlo sampling through 2D visualisations that clearly demonstrate the improved spread. The section concludes with an

experiment that underscores the practical advantages of these alternative approaches for creating a robust initial one-shot training set.

### 5.1.1 Latin Hypercube Sampling

Latin Hypercube Sampling (LHS) [26] is a well-established space-filling design that addresses the shortcomings of random (Monte Carlo) sampling by providing a more uniform representation of the input space. In LHS, each input variable (typically normalised to the interval  $[0, 1]$ ) is divided into  $n$  equally spaced intervals. One sample is drawn from each interval, and a random permutation is applied independently for each dimension to combine these samples into a  $d$ -dimensional design.

Mathematically, let  $n$  be the number of samples and  $d$  be the number of dimensions. For each variable  $j \in \{1, \dots, d\}$ , the range is partitioned into  $n$  intervals of width  $1/n$ . For sample  $i \in \{1, \dots, n\}$  and dimension  $j$ , generate a random number  $u_{ij} \sim U(0, 1)$  and let  $\pi_j$  be a random permutation of the set  $\{1, 2, \dots, n\}$ . The value of the  $j$ th component of the  $i$ th sample is computed as,

$$x_{ij} = \frac{\pi_j(i) - 1 + u_{ij}}{n}.$$

Figure 5.1 illustrates the improved distribution provided by LHS compared to simple random sampling.

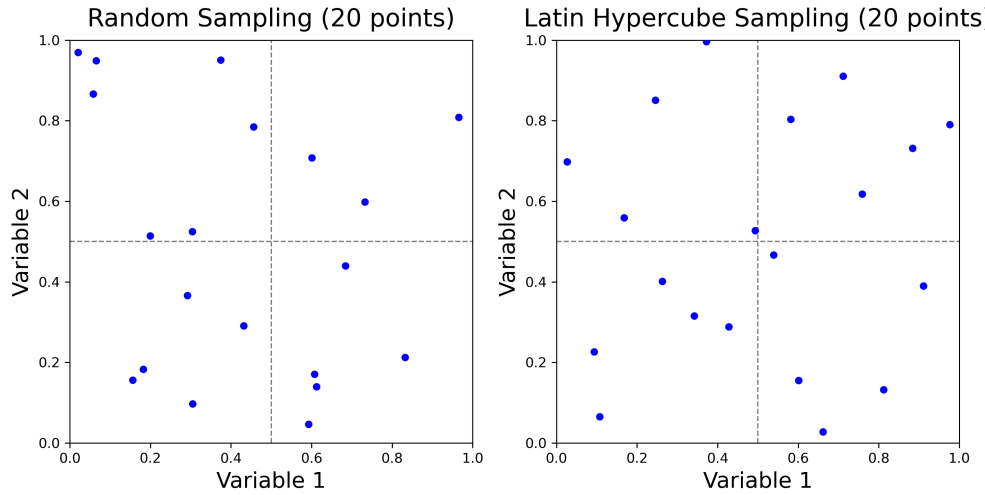


Figure 5.1: Comparison of Random (Monte Carlo) sampling versus LHS.

### 5.1.2 Sobol Sequences

Sobol sequences are a type of low-discrepancy sequence that are designed to uniformly fill the unit hypercube [27]. Unlike Monte Carlo sampling, which can suffer from clustering and uneven coverage, Sobol sequences are generated by a deterministic algorithm that minimises discrepancy across the input space. Although



the full algorithm involves the use of direction numbers and bitwise operations, the key idea is that the sequence  $\{\mathbf{x}_i\}_{i=1}^n$  is constructed so that the number of points in any sub-region of the unit hypercube closely approximates the expected number under a perfectly uniform distribution. Figure 5.2 shows a comparison between a Monte Carlo sample and a sample obtained using Sobol sequences, demonstrating the more uniform distribution achieved with the latter.

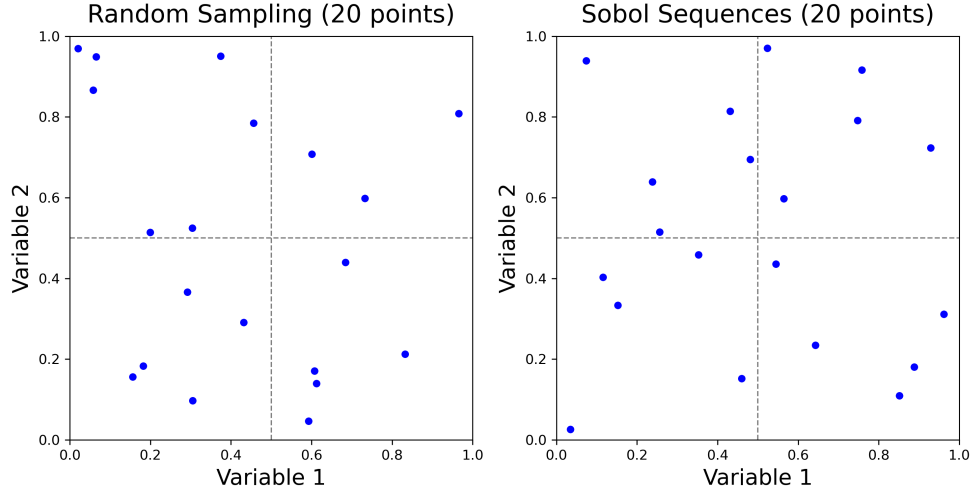


Figure 5.2: Comparison of Random (Monte Carlo) Sampling versus Sobol Sequences.

### 5.1.3 Comparing One-Shot Selection Methods

Figure 5.3 shows the contour plots for the Branin and Six-Hump Camel functions. These benchmark functions are standard test problems in global optimisation [28] because they present challenging landscapes for evaluating different sampling and optimisation methods.

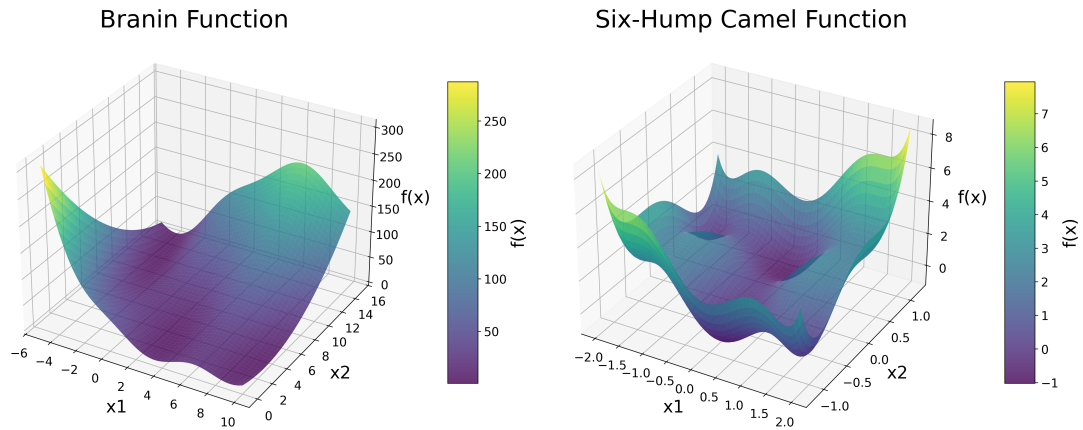


Figure 5.3: Combined contour plots for the Branin and Six-Hump Camel functions, illustrating the sample coverage.

The experiment we conducted evaluates the three one-shot sampling strategies described above on the Branin and Six-Hump Camel functions. For each strategy,

15 training points are sampled within the function’s domain, and a GP is trained on this dataset, then evaluated on a fixed set of 1000 uniformly distributed test points using RMSE. This process is repeated for 50 independent trials per strategy to account for variability. Figure 5.4 summarises the RMSE distributions across trials.

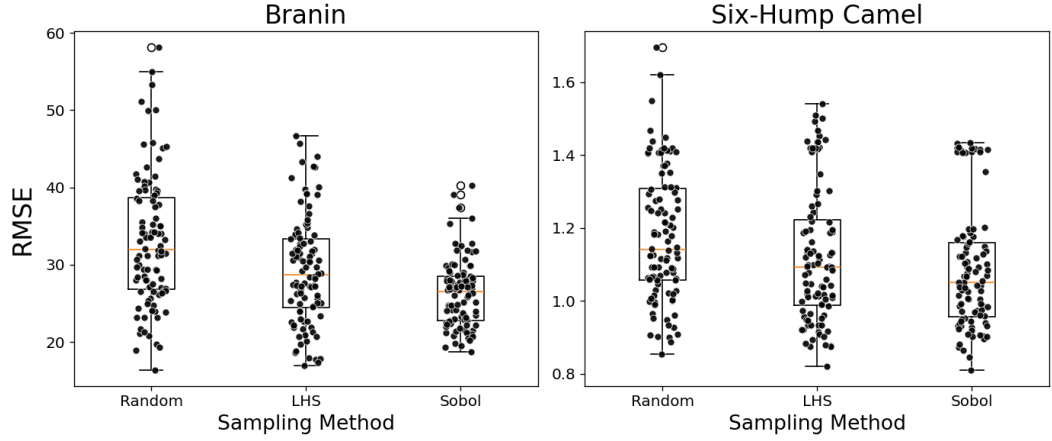


Figure 5.4: Boxplots of RMSE for different sampling methods.

Table 5.1 confirms that both space-filling methods significantly outperform random sampling across both functions (all p-values less than 0.05 for LHS/Sobol vs. Random). Sobol exhibits a decisive advantage over LHS on the smoother Branin function ( $p = 2.2\text{e-}4$ ), likely due to its design better approximating broad trends. However, on the multimodal Six-Hump Camel surface, Sobol’s edge over LHS diminishes ( $p = 0.154$ ). This contrast displays how the efficiency of space-filling strategies depends on function geometry: Sobol excels in smoother landscapes, while LHS remains competitive in complex landscapes where even structured sampling may miss critical features. However, both methods consistently surpass random sampling, reaffirming their value in GP initialisation.

Table 5.1: Paired t-test results comparing RMSE between sampling methods.

| Function              | Comparison       | t-statistic | p-value   |
|-----------------------|------------------|-------------|-----------|
| <b>Branin</b>         | LHS vs. Random   | -3.388      | 1.012e-03 |
|                       | Sobol vs. Random | -7.063      | 2.291e-10 |
|                       | Sobol vs. LHS    | -3.836      | 2.199e-04 |
| <b>Six-Hump Camel</b> | LHS vs. Random   | -2.142      | 3.462e-02 |
|                       | Sobol vs. Random | -3.808      | 2.434e-04 |
|                       | Sobol vs. LHS    | -1.438      | 1.535e-01 |

## 5.2 Sequential Training Point Selection

We will now turn our focus to the GP model-based sequential point selection, also referred to as active learning. Suppose there is some hard-to-evaluate function  $g$

(for example, this could be the result from an expensive scientific experiment). Our goal is to harness the power of GPs to model  $g$  efficiently (this is a similar setting to Bayesian Optimisation which we discuss in detail in Chapter 6). Active learning is an iterative process in which new training points for the GP are intelligently selected to boast the maximum improvement in the model's performance, while minimising the required number of expensive function evaluations. Concretely, this process consists of the following steps [29]:

**Step 1:** Select an initial *seed* set  $X_{n_0}$  consisting of  $n_0$  points. Then evaluate  $g$  at these points to obtain the corresponding outputs  $Y_{n_0}$  to form the initial dataset  $D_{n_0} = (X_{n_0}, Y_{n_0})$ . We ideally want  $n_0$  as large as possible within the constraints imposed on us.

**Step 2:** Fit a surrogate GP model to the dataset  $D_n$  and update hyperparameters in each new iteration via Maximum Likelihood Estimation (MLE).

**Step 3:** Solve a sequential criterion  $J(x)$  based on the fitted model from Step 2 to determine the next training point. Specifically,

$$x_{n+1} = \arg \max_x J(x) \mid D_n.$$

The function  $J(x)$  quantifies how informative a candidate point  $x$  is likely to be. An example will be given in 5.2.1.

**Step 4:** Compute the corresponding output  $y_{n+1} = g(x_{n+1})$ .

**Step 5:** Update the dataset  $D_{n+1} = D_n \cup (x_{n+1}, y_{n+1})$ ; Set  $n \leftarrow n + 1$  and repeat from Step 2 as many times as possible within the constraints.

In Step 3, different choices for  $J(x)$  lead to different active learning designs. One such option for choosing the next training point is Active Learning MacKay.

### 5.2.1 Active Learning MacKay

Active Learning MacKay (ALM) is a widely applicable sequential design for GPs [29]. ALM chooses the next point by maximising the predictive variance  $\sigma_n^2(x)$  at points  $x$ . Formally, we choose  $J(x) = \sigma_n^2(x)$  so that,

$$x_{n+1} = \arg \max_x \sigma_n^2(x).$$

This criterion directs sampling towards the most uncertain regions of the input space.

When considering ALM, Step 3 (from above) requires computing the predictive variance  $\sigma_n^2(x)$  across the design space. It is worth noting that the variance function can have multiple local maxima, and the number of local maxima will likely grow with the number of sample points. Thus optimisation can sometimes

be challenging in this non-convex space. Multiple restarts and local optimisation to high variance at individual points can be used to address this, however this technique might not be feasible if  $g$  is especially expensive to compute. We used the L-BFGS-B optimisation method [30] for this local optimisation search.

We now illustrate the first two iterations of ALM in an example with an initial seed set of 12 points generated using LHS. Figure 5.5 demonstrates this process. The left subplot shows arrows indicating the outcomes of multi-start local optimisations, converging toward high-variance regions. The cross marker denotes the selected  $x_{n+1}$ . The right subplot shows the second iteration after updating the GP model with the newly acquired data point in the left subplot. It is common for several arrows to terminate at the same location (this is likely due to a local maximum in variance). We can see that at this early stage of sequential design, the new points we selected are most likely positioned along the edge of the input space, as this is where the LHS has not placed any initial seed points, and so variance will be highest.

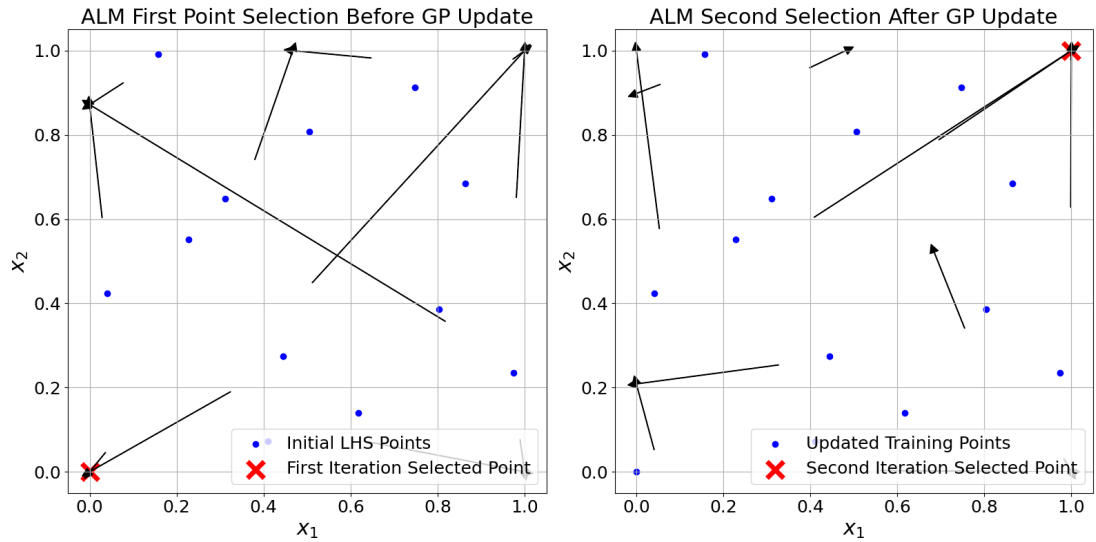


Figure 5.5: First iteration of ALM search (left) and second iteration of ALM search (right). Each arrow represents an origin and outcome of multi-start exploration of predictive variance. The decided variance-maximising location is labeled with a cross sign.

### 5.2.2 Evaluating the Performance of ALM

To test the effectiveness of ALM as a design choice, we performed the following experiment. The experiment evaluates whether combining Sobol sequences as a seed set (the top-performing one-shot strategy from the previous test) with ALM improves GP accuracy over the one-shot methods compared previously on the Branin and Six-Hump Camel functions. For the hybrid strategy, 5 initial Sobol points seed the design, followed by 10 adaptive selections using MacKay’s variance maximisation, totalling 15 training points. The one-shot strategies also had 15 training points sampled as before. All methods train a GP on their respective

15-point datasets and are tested on a fixed 1000-point grid, with RMSE computed over 100 trials. Figure 5.6 compares RMSE distributions.

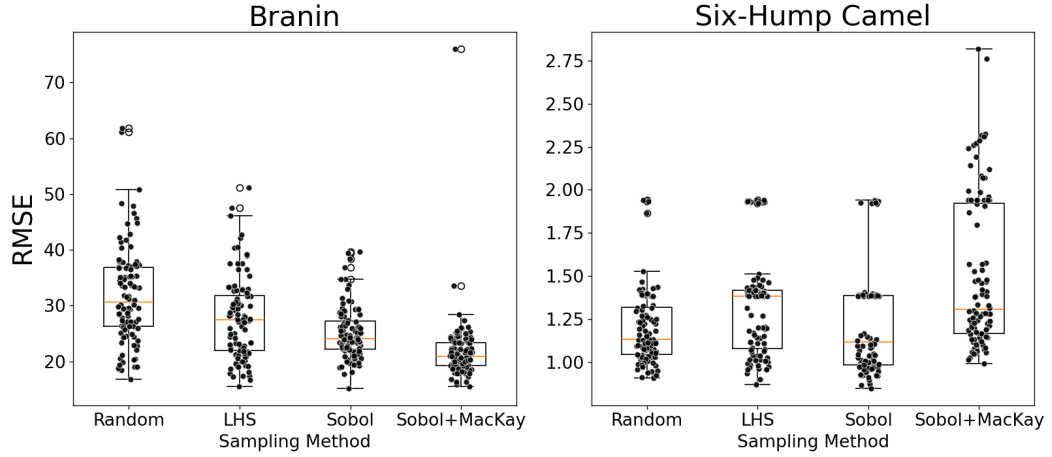


Figure 5.6: Boxplots of RMSE for different sampling methods.

Table 5.2 reveals a critical trade-off: the Sobol+MacKay hybrid strategy significantly outperforms all one-shot methods on the smooth Branin function, but significantly underperforms on the multimodal Six-Hump Camel. For smooth functions like Branin, MacKay’s variance-driven exploitation effectively refines Sobol’s initial space-filling design, prioritising regions of high uncertainty that correlate with under-sampled areas. However, on the Six-Hump Camel function the same strategy risks overfitting to high-variance regions, often sampling near steep slopes rather than global minima.

Table 5.2: Paired t-test results comparing RMSE between Sobol+MacKay and other sampling methods.

| Function              | Comparison             | t-statistic | p-value   |
|-----------------------|------------------------|-------------|-----------|
| <b>Branin</b>         | Sobol+MacKay vs Random | -9.10       | 9.957e-15 |
|                       | Sobol+MacKay vs LHS    | -6.30       | 8.319e-09 |
|                       | Sobol+MacKay vs Sobol  | -4.04       | 1.061e-04 |
| <b>Six-Hump Camel</b> | Sobol+MacKay vs Random | 5.81        | 7.467e-08 |
|                       | Sobol+MacKay vs LHS    | 3.07        | 2.736e-03 |
|                       | Sobol+MacKay vs Sobol  | 5.10        | 1.667e-06 |

This suggests hybrid methods excel in smooth or well-conditioned settings, but their reliability diminishes in complex landscapes. Unless prior knowledge of smoothness exists, one-shot designs provide safer initialisation by ensuring the uniform spread of the training points. The code for this experiment is included in Appendix A.4 for reference,

# Chapter 6

## Bayesian Optimisation

Bayesian optimisation is a method for efficiently finding the optimum of functions that are expensive or time-consuming to evaluate, especially when these functions lack an analytical expression and can only be examined through individual evaluations. Instead of evaluating every possible point, this approach builds a probabilistic approximation of the function, commonly using a GP, which provides both predicted values and an estimate of uncertainty at each point. An accompanying mechanism, known as an acquisition function, leverages this information to decide where to sample next by considering regions that show promise and areas where the model is uncertain. One notable application of this strategy is in hyperparameter tuning, where evaluating every possible configuration is impractical [31].

### 6.1 The Bayesian Approach to Model Selection:

#### 6.1.1 Model Selection

Before we consider using Bayesian optimisation, we should first consider the model selection problem from a Bayesian perspective. Model selection is the process of selecting and setting parameters, hyperparameters, and model choices. For example, when considering a GP model, if we choose to use an RBF kernel, we have seen in Section 2.2.1 that altering the length scale significantly can cause our inference to be drastically different. Similarly, for the actual model, we could change the covariance function that we use in the fitting process, doing so would encode different beliefs about the data in our model. It is worth noting here that in GPs, although the model is non-parametric, the ‘parameters’ in this context are typically seen as the latent function outputs at the training points.

When we consider model selection more generally, we usually have a layered selection setup. At the first (lowest) level of our model, we have the parameter values (e.g. coefficients of a linear regression model). At the next level we

consider the hyperparameters of the model (e.g. the learning rate of a neural network). Finally, we consider a discrete set of models that we wish to compare and select from (e.g. the choice of covariance function for a GP or the number of features for a linear regression model).

For creating the model selection posterior formulae we follow the methodology of [1]. For the first level, we have the parameter posterior, which is given by Bayes' rule to be,

$$p(\mathbf{w}|\mathbf{y}, \boldsymbol{\theta}, X, \mathcal{M}_i) = \frac{p(\mathbf{y}|\mathbf{w}, X, \mathcal{M}_i)p(\mathbf{w}|\boldsymbol{\theta}, \mathcal{M}_i)}{\int p(\mathbf{y}|\mathbf{w}, X, \mathcal{M}_i)p(\mathbf{w}|\boldsymbol{\theta}, \mathcal{M}_i)d\mathbf{w}},$$

where  $X$  is the design matrix for the inputs,  $\boldsymbol{\theta}$  is the vector of hyperparameters, and  $\mathcal{M}_i$  is the discrete set of models we wish to consider, indexed by  $i$ .

At the second level we have the hyperparameter posterior,

$$p(\boldsymbol{\theta}|\mathbf{y}, X, \mathcal{M}_i) = \frac{p(\mathbf{y}|\boldsymbol{\theta}, X, \mathcal{M}_i)p(\boldsymbol{\theta}|\mathcal{M}_i)}{\int p(\mathbf{y}|\boldsymbol{\theta}, X, \mathcal{M}_i)p(\boldsymbol{\theta}|\mathcal{M}_i)d\boldsymbol{\theta}}.$$

Finally, at the top level we have the model posterior,

$$p(\mathcal{M}_i|\mathbf{y}, X) = \frac{p(\mathbf{y}|X, \mathcal{M}_i)p(\mathcal{M}_i)}{\sum_{i=1}^N p(\mathbf{y}|X, \mathcal{M}_i)p(\mathcal{M}_i)},$$

where we note that since  $\mathcal{M}_i$  is a discrete set we take the sum over its entries, thus there is no integral involved here. It is also worth noting that typically  $p(\mathcal{M}_i)$  is taken to be an uninformative prior as we do not want to favor one model over another apriori unless we have explicit evidence that this is the case.

The term  $p(\mathbf{y}|\boldsymbol{\theta}, X, \mathcal{M}_i)$  is known as the marginal likelihood with respect to the hyperparameters and instead of attempting to evaluate the integral in the normalising constant of the hyperparameter posterior, we may try to maximise the marginal likelihood with respect to  $\boldsymbol{\theta}$  instead [1]. Doing this can incur a chance of overfitting the model, but (as we will see) this is not a significant issue since the marginal likelihood actually penalises unnecessarily complex models. When deciding on prior distributions, we must also be careful to allow them to be general enough to explain the data, since if we consider a very strict prior that does not allow our model to describe our data, the inferences we obtain will likely be incorrect.

### 6.1.2 Hyperparameter Selection for GPs

A lot of the integrals stated above are often analytically intractable and difficult to approximate, however GPs with a Gaussian noise factor are a pleasant exception to this. In this case the marginal likelihood is given by,

$$\log(p(\mathbf{y}|\boldsymbol{\theta}, X)) = -\frac{1}{2} \log |K_y| - \frac{1}{2} \mathbf{y}^\top K_y^{-1} \mathbf{y} - \frac{n}{2} \log(2\pi),$$

([32] chapter 3) where  $K_y = K_f + \sigma_n^2 I$  is the covariance matrix of the noisy samples,  $\mathbf{y}$ , and  $K_f$  is covariance matrix of the noiseless function values,  $f(\mathbf{x})$ . We now notice that each term of this expression has an easily understood meaning. The first term is a complexity penalty term that remains small in magnitude for simple models and penalises overly complex models as the log determinant increases. The second term is our data-fit term that increases for more complex models and decreases for simpler models that explain less of the already observed data (note that the final term is simply a constant). We see this illustrated in Figure 6.1 where the log-likelihood peaks at around  $\ell = 0.8$ . The data-fit term decreases rapidly when we pass a value of  $\ell = 1$  as the model does not fit the data well at all for larger length scales. The data term increases far more slowly when decreasing the length scale from  $\ell = 0.8$  as the model does not capture noticeably more variation of the observed data by becoming more complex past this point. The penalty term increases as we increase the length scale (leading to a simpler model) and controls the marginal likelihood's shape when the data-fit term stops improving noticeably for smaller length scales.

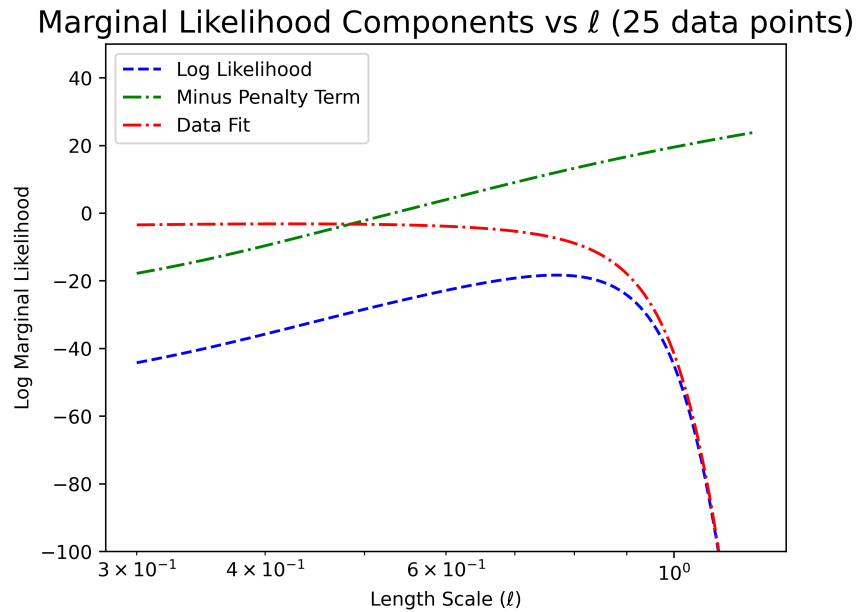


Figure 6.1: Marginal likelihood decomposition for a GP with an RBF kernel. The plot shows the log-likelihood and its components (excluding the constant term) as a function of  $\ell$  where there are 25 training data points.

Figure 6.2 shows the shape of the log marginal likelihood as a function of the sample size, and we see that when we have more data the optimum becomes more peaked.

Now that we have covered the theory required, we consider optimising the hyperparameters by using the marginal likelihood metric. If we wish to use a gradient based optimiser then we can determine the derivatives of the marginal likelihood



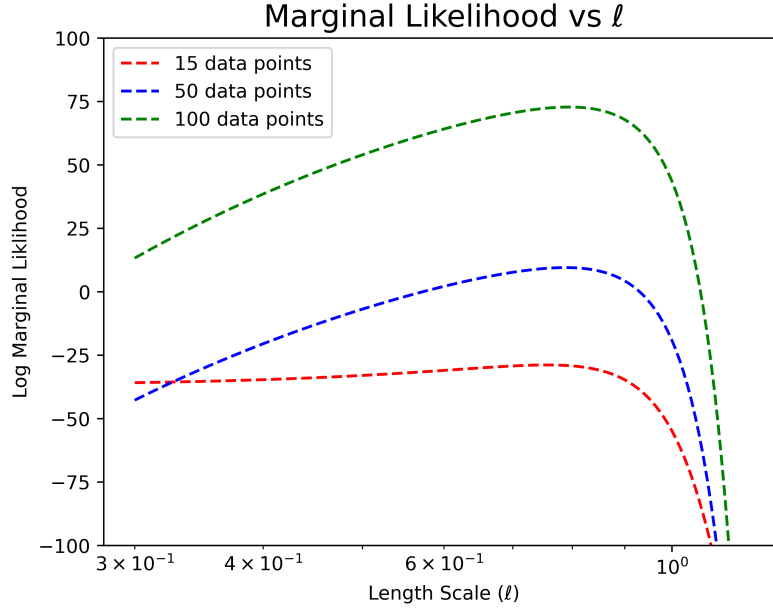


Figure 6.2: Marginal likelihood decomposition for a GP with an RBF kernel. This plot visualises the marginal likelihood against the length scale,  $\ell$ , for different sample sizes of  $\mathbf{y}$  (where  $\sigma_n = 0.1$ ).

(stated above) with respect to the hyperparameters by using the fact [33] that,

$$\frac{\partial}{\partial \theta} K^{-1} = -K^{-1} \frac{\partial K}{\partial \theta} K^{-1} \quad \text{and} \quad \frac{\partial}{\partial \theta} \log |K| = \text{tr} \left( K^{-1} \frac{\partial K}{\partial \theta} \right),$$

for positive definite matrices  $K$  to obtain,

$$\begin{aligned} \frac{\partial}{\partial \theta_i} (\log(p(\mathbf{y}|\boldsymbol{\theta}, X))) &= \frac{\partial}{\partial \theta_i} \left[ -\frac{1}{2} \log |K_y| - \frac{1}{2} \mathbf{y}^\top K_y^{-1} \mathbf{y} - \frac{n}{2} \log(2\pi) \right] \\ &= -\frac{1}{2} \text{tr} \left( K_y^{-1} \frac{\partial K_y}{\partial \theta_i} \right) + \frac{1}{2} \mathbf{y}^\top K_y^{-1} \frac{\partial K_y}{\partial \theta_i} K_y^{-1} \mathbf{y}. \end{aligned}$$

We can then construct a gradient based optimiser in the usual way from here. The computational cost of computing these derivatives is determined primarily by the inversion of the matrix  $K_y$  which takes  $\mathcal{O}(n^3)$  for positive definite matrices [1] (where  $n$  is the sample size of the model). Once we have computed this term (which we note we only need to do once to get all of the derivatives for a given iteration of our optimiser) the remaining cost of calculating the gradient of each hyperparameter is  $\mathcal{O}(n^2)$  [1]. This method will be used in the adaptive Bayesian optimisation section later in this chapter to find the optimal hyperparameters for the *surrogate* model (surrogate models will be defined formally in the next section but for now you should consider it as a sort of ‘submodel’ of the main model). The surrogate model will generally have far less training points than the main model so computing the derivatives within the surrogate model will speed up the process considerably (or even make it feasible) compared to if we decided

to face optimising the objective function head-on with this approach.

The marginal likelihood can have multiple local optima which can cause issues if one is not careful, however this is typically not too troublesome when optimising over the hyperparameter space. To ensure a higher level of certainty, it is common to restart at various random initialisations for  $\theta$  and then take the best result to be the optimum of the marginal likelihood over the parameter space. To give a visual representation of the marginal likelihood's behavior as a function of the hyperparameters (which is what we will be optimising over with our gradient based optimiser) Figure 6.3 illustrates an example using a GP with an RBF kernel where  $\sigma_f^2 = 3.5$ . We can see that the hyperparameter space for this example does indeed have a clearly defined optimum. Since the data used here was the same as in Figure 6.1, one can draw comparisons between the two figures, since as we increase the length scale from the optimum in both figures, the marginal likelihood decreases drastically since the model does not fit the data well anymore.

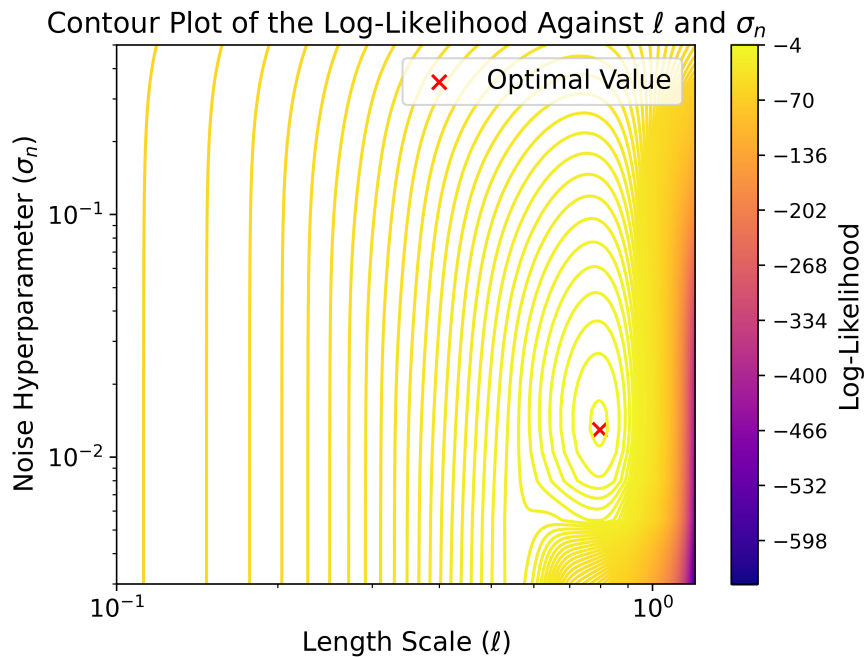


Figure 6.3: This plot shows the marginal likelihood against the length scale,  $\ell$ , and the noise variance,  $\sigma_n$  for a GP using an RBF kernel. The red cross is the maximum value of the marginal likelihood.

## 6.2 Theory of Bayesian Optimisation

### 6.2.1 Constructing the Surrogate Model

In the first step of Bayesian optimisation, a surrogate model is constructed to approximate the unknown objective function based on available data from previous evaluations. GPs are commonly used due to their flexibility and ability to provide

both mean predictions and uncertainty estimates [34]. Figure 6.4 illustrates the initial surrogate model constructed using a GP.

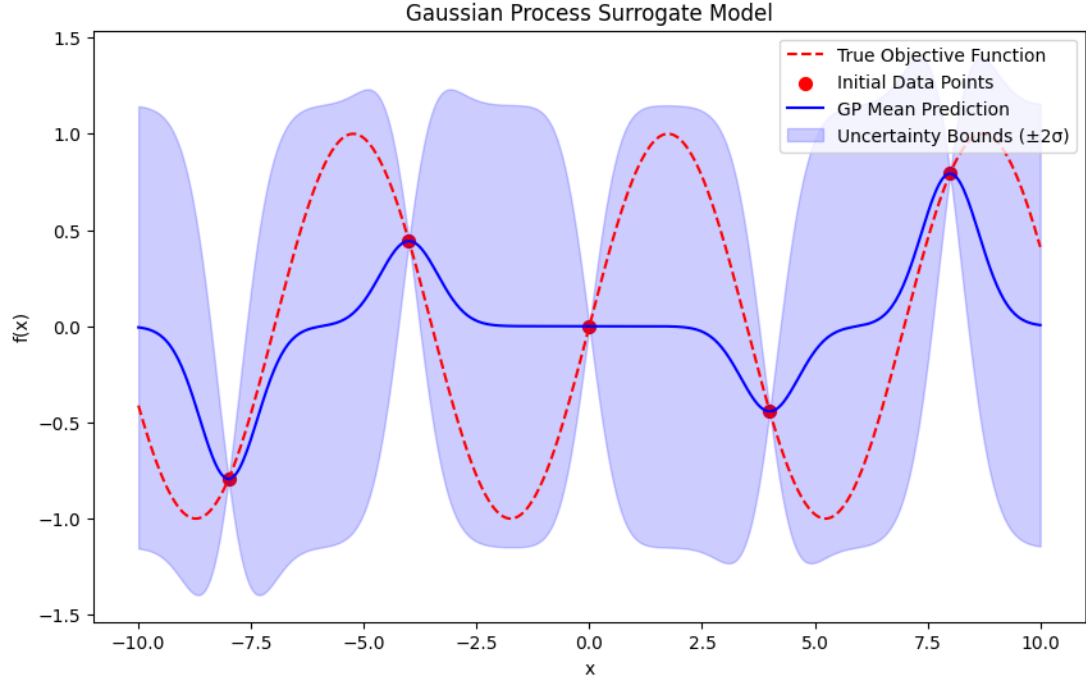


Figure 6.4: Initial GP surrogate model with uncertainty estimates.

The figure highlights several key features. The red dashed line represents the true objective function—this is the function being optimised, even though it remains unknown in practice. The red dots mark the initial data points at which the objective function has been evaluated. The solid blue line shows the mean prediction produced by the GP, which forms the surrogate model’s approximation of the unknown function. The blue shaded region indicates the uncertainty bounds around this mean, typically corresponding to two standard deviations, capturing both the model’s current understanding and its confidence in untested areas.

It is also evident that, in regions away from the data points, the surrogate model defaults to a prior value of zero, indicating that little is known about these areas. A small level of observational noise is present near the data points, while most of the uncertainty is observed in regions without any collected data.

### 6.2.2 Defining and Maximising the Acquisition Function

Once the surrogate model is in place, the next step in Bayesian optimisation is to define an acquisition function. This function leverages the predictions of the GP to determine where to evaluate next, balancing exploration, which samples regions where the model is uncertain, with exploitation, which focuses on areas where the objective function is likely to improve on the current best value.

In this work, we adopt the Expected Improvement acquisition function, as it

provides a clear measure of the potential gain from sampling a particular point. Expected Improvement quantifies the expected improvement over the best observed outcome, taking into account both the predicted mean and uncertainty at each candidate point. The formal definition of Expected Improvement is given below and illustrated in Figure 6.5.

**Definition 5.** [35] For a given best observed value  $y^* \in \mathbb{R}$  and a GP prediction at input  $x$  with mean  $\hat{f}(x)$  and standard deviation  $\sigma(x)$ , the Expected Improvement (EI) is defined as,

$$EI(x) = (y^* - \hat{f}(x)) \Phi\left(\frac{y^* - \hat{f}(x)}{\sigma(x)}\right) + \sigma(x) \phi\left(\frac{y^* - \hat{f}(x)}{\sigma(x)}\right),$$

where  $\phi(\cdot)$  and  $\Phi(\cdot)$  denote the standard normal density and cumulative distribution functions, respectively (with  $\sigma(x) > 0$ ).

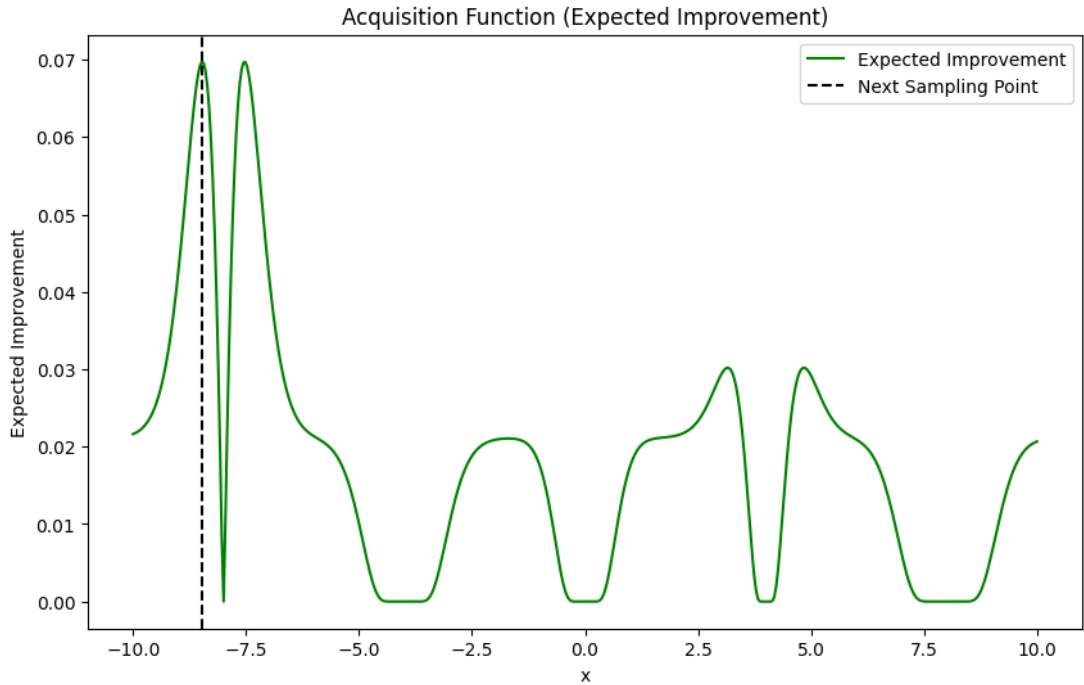


Figure 6.5: Expected Improvement acquisition function indicating the next sampling point.

Figure 6.5 depicts the Expected Improvement values across the domain. The green curve shows the expected improvement at each candidate point, with its peaks indicating regions where a significant improvement over  $y^*$  is likely. The vertical dashed line marks the maximum of the Expected Improvement curve, suggesting the most promising location for the next evaluation. This approach effectively directs the optimisation process by considering both the model's predictions and the associated uncertainty provided by the surrogate.

### 6.2.3 Iterative Optimisation Process

Bayesian optimisation proceeds iteratively, refining both the surrogate model and the acquisition function with each new observation. The process unfolds in three stages. First, the selection phase uses the acquisition function to identify the next input point to sample. Second, the evaluation phase computes the objective function at this new point, adding an extra data point to the surrogate’s dataset. Third, the update phase integrates this new data into the surrogate model. This cycle is repeated until a predefined stopping criterion, such as a computational budget, iteration limit, or convergence threshold is met.

Figure 6.6 visualises the evolution of the surrogate model across multiple iterations. The figure overlays mean predictions from iterations 0, 2, 4, 6, and 8, each distinguished by a unique color. Black dots denote all sampled points accumulated during the process, illustrating the algorithm’s progressive focus on regions near the optimum. The uncertainty bounds, depicted as shaded regions around each mean prediction, gradually tighten in areas where data is collected. For reference, the true objective function is shown as a red dashed line. Over time, the surrogate model converges toward the true function, demonstrating Bayesian optimisation’s ability to efficiently allocate evaluations to high-potential regions while reducing uncertainty.

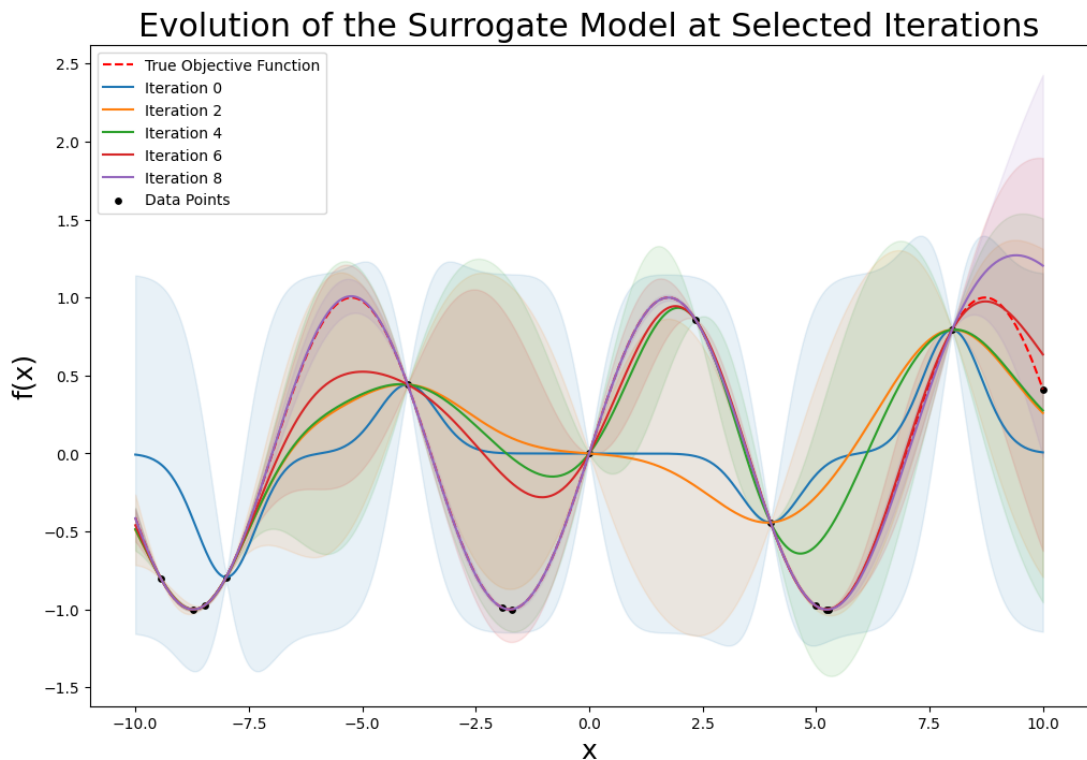


Figure 6.6: Evolution of the surrogate model over selected iterations in the Bayesian optimisation process.

### 6.3 Adaptive Bayesian Optimisation

In the conventional Bayesian optimisation framework, a single set of hyperparameters for the surrogate model, denoted as  $\boldsymbol{\theta}$  (typically selected by maximising the marginal likelihood, see Section 6.1.2), is held fixed throughout the optimisation process. Note that these surrogate model hyperparameters,  $\boldsymbol{\theta}$ , are distinct from the input locations,  $\boldsymbol{x}$ , that are queried by the optimisation algorithm. The correct choice of hyperparameters for the surrogate model will guide the algorithm to the most optimal solution.

Although fully Bayesian approaches address hyperparameter uncertainty by integrating over  $\boldsymbol{\theta}$  [36], they are computationally demanding and may converge slowly in dynamic settings. Inspired by the iterative refinement of the Expectation-Maximisation algorithm [37], our proposed strategy updates the surrogate model's hyperparameters at every iteration using gradient-based optimisation. Before the acquisition function selects the next input location, the surrogate model is given all current objective function values and their corresponding input location(s). If this becomes too computationally expensive, the strategy can be altered so that optimisation occurs every  $n^{th}$  iteration. Using this information, it updates its hyperparameters  $\boldsymbol{\theta}$  via gradient descent, allowing the algorithm to capture more complex patterns and adjust for any suboptimal initialisation.

Although this iterative update increases computational cost, our work restricts the total number of iterations so that updating  $\boldsymbol{\theta}$  at every step is feasible. Convergence is declared either when the maximum number of iterations is reached or when both the surrogate model's hyperparameters and the acquisition function values change only negligibly, indicating that further exploration is unlikely to yield significant improvements.

The full details of this adaptive approach are summarised in Algorithm 1.

**Algorithm 1** Adaptive Bayesian Optimisation

- 
- 1: **Input:** Objective function  $\mathcal{L} : \mathbb{R}^d \rightarrow \mathbb{R}$
  - 2: Choose an initial point  $\mathbf{x}_0 \in \mathbb{R}^d$  and compute  $y_0 = \mathcal{L}(\mathbf{x}_0)$
  - 3: Initialise the data set:  $\mathcal{S}_0 = \{(\mathbf{x}_0, y_0)\}$
  - 4: **for**  $t = 1, 2, \dots, T_{\max}$  **do**
  - 5:     **Update surrogate model:** Optimise hyperparameters by solving

$$\boldsymbol{\theta}_t = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \mathcal{J}(\boldsymbol{\theta}; \mathcal{S}_{t-1}),$$

where  $\mathcal{J}$  is the loss function for the surrogate model.

- 6:     **Compute acquisition function:** Define

$$A_t(\mathbf{x}) = A(\mathbf{x}; \boldsymbol{\theta}_t)$$

for all  $\mathbf{x} \in \mathbb{R}^d$ .

- 7:     **Select next point:** Choose

$$\mathbf{x}_t = \underset{\mathbf{x} \in \mathbb{R}^d}{\operatorname{argmax}} A_t(\mathbf{x}).$$

- 8:     **Evaluate objective:** Compute  $y_t = \mathcal{L}(\mathbf{x}_t)$ .
  - 9:     **Update data set:**  $\mathcal{S}_t = \mathcal{S}_{t-1} \cup \{(\mathbf{x}_t, y_t)\}$ .
  - 10:    **if** Convergence: either  $t = T_{\max}$  or  $\Delta\boldsymbol{\theta}_t$  and  $\Delta A_t$  below threshold **then**
  - 11:       **Break**
  - 12:    **end if**
  - 13: **end for**
  - 14: **Return:**  $\mathbf{x}^* = \underset{(\mathbf{x}, y) \in \mathcal{S}_t}{\operatorname{argmin}} y$
- 

Here we provide a concise explanation of each step in Algorithm 1. The optimisation begins with an objective function  $\mathcal{L}(\mathbf{x})$ , where  $\mathbf{x} \in \mathbb{R}^d$  denotes the  $d$ -dimensional hyperparameter vector for the main model. The surrogate dataset is initialised with an initial configuration  $\mathbf{x}_0$  and its evaluation  $\mathcal{L}(\mathbf{x}_0)$ , i.e.,

$$\mathcal{S} \leftarrow \{(\mathbf{x}_0, \mathcal{L}(\mathbf{x}_0))\}.$$

These initial configurations should be chosen using a sensible point selection technique (see Section 5.1). At each iteration  $t$  (from 1 to  $T_{\max}$ ), the algorithm first updates the surrogate model using the current dataset  $\mathcal{S}_{t-1}$ . In this update, the surrogate refines its own hyperparameters  $\boldsymbol{\theta}_t$  via gradient descent (see Section 6.1.2). The updated model is then used to compute an acquisition function  $A(\mathbf{x})$  over the input space given the current set of hyperparameters for the surrogate GP, and the next candidate is selected.

After evaluating the objective function at  $\mathbf{x}_t$ , the pair  $(\mathbf{x}_t, y_t)$  is added to  $\mathcal{S}$ . Convergence is assessed either when the maximum number of iterations  $T_{\max}$  is

reached or if changes in both the surrogate hyperparameters and the acquisition function values become negligible. Finally, the optimal hyperparameters are determined by selecting the configuration in  $\mathcal{S}$  that minimises  $\mathcal{L}(\mathbf{x})$ ,

$$\mathbf{x}^* = \arg \min_{\mathbf{x} \in \mathcal{S}} \mathcal{L}(\mathbf{x}).$$

This configuration is returned as the final outcome of the optimisation process.

### 6.3.1 Optimising a Noisy 1D Unknown Function

In this example, our goal is to find the minimum of an unknown synthetic function that is intentionally designed to incorporate noise, representing a scenario where sampling is costly and noisy. Bayesian optimisation is widely utilised to identify optima in settings where each function evaluation is computationally expensive [38]; this example demonstrates why adaptive Bayesian optimisation is advantageous in such cases. As described in Algorithm 1 that at each iteration the candidate hyperparameter configuration  $\theta_t$  is selected via a loss function - specifically, the Negative Log-Likelihood optimised using Stochastic Gradient Descent. Here, we interpret the algorithm's notation as follows: we define the surrogate GP's kernel hyperparameters as,

$$\theta = (\ell, \sigma_f),$$

where we chose to use an RBF kernel. The objective function  $\mathcal{L}(\theta)$  is given by the synthetic noisy function, and by minimising  $\mathcal{L}$  we aim to determine the input location corresponding to its minimum.

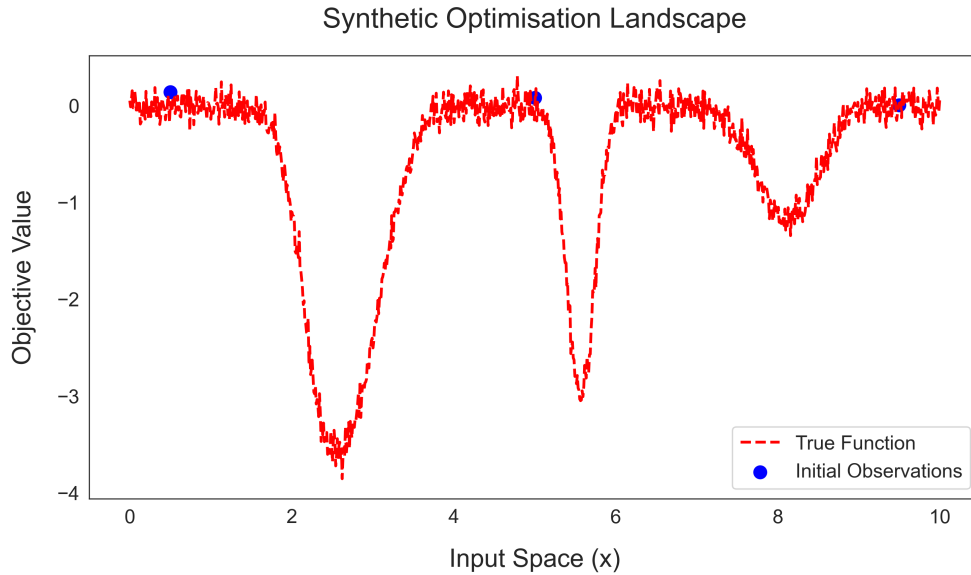


Figure 6.7: Noisy optimisation landscape with three initial observations.

Figure 6.7 establishes the optimisation challenge, the initial observations were chosen to replicate the spread of an initialisation with LHS.



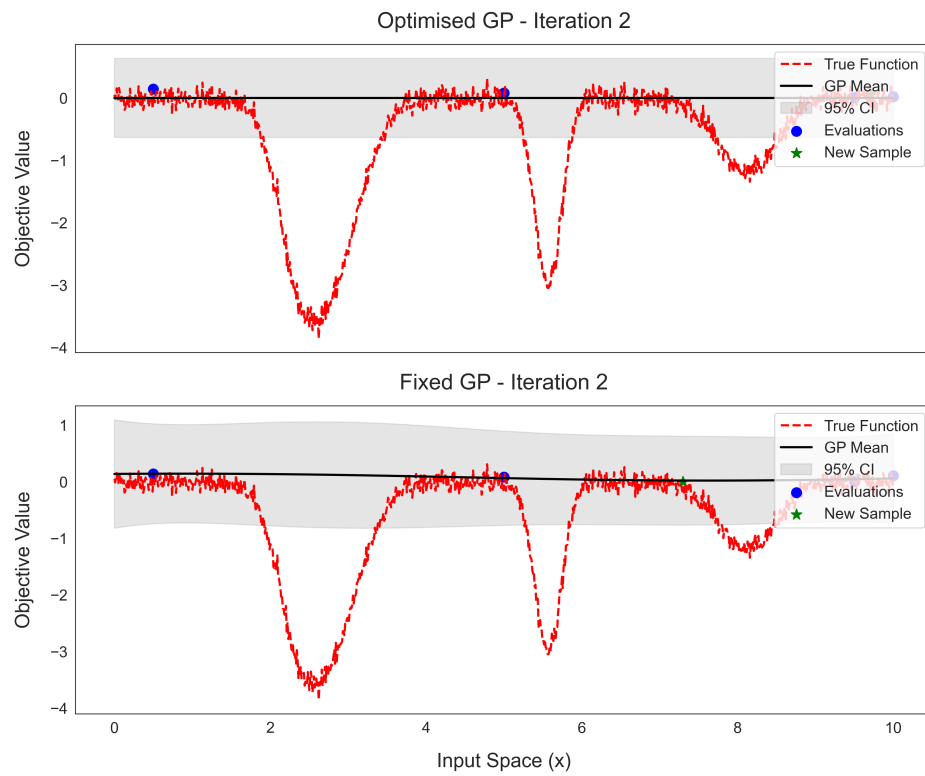


Figure 6.8: Early-stage comparison after two iterations.

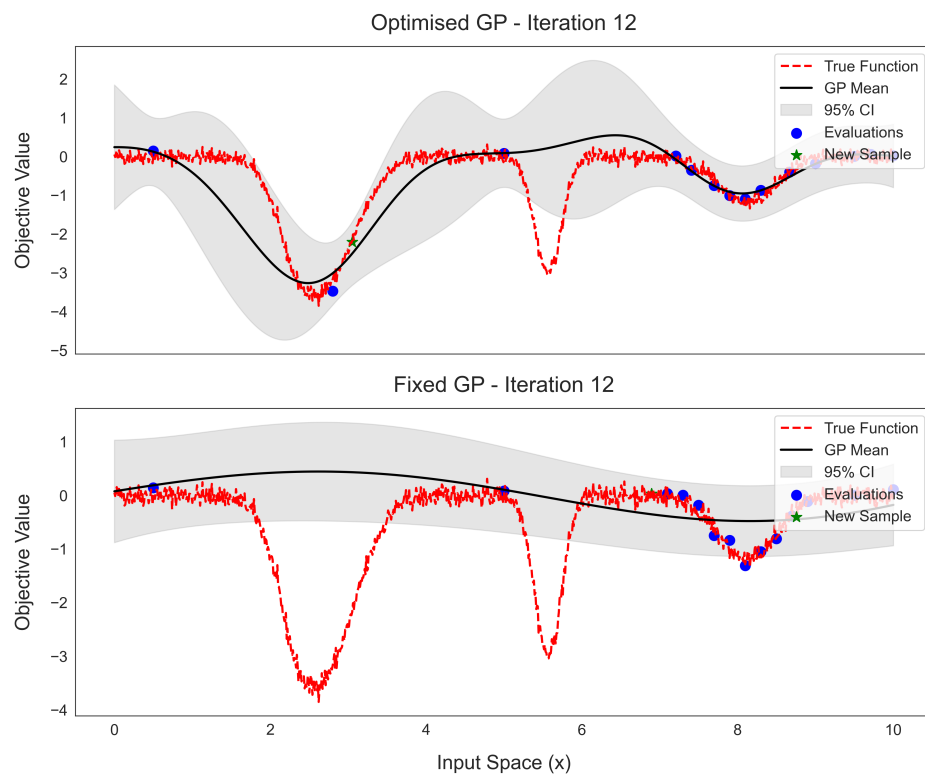


Figure 6.9: Late-stage comparison after twelve iterations.

The iterative comparisons (Figures 6.8-6.9) reveal the advantages of hyperparameter adaptation. The optimised surrogate GP has dynamically reduced its lengthscale, enabling resolution of local features, while simultaneously adjusting its noise variance.

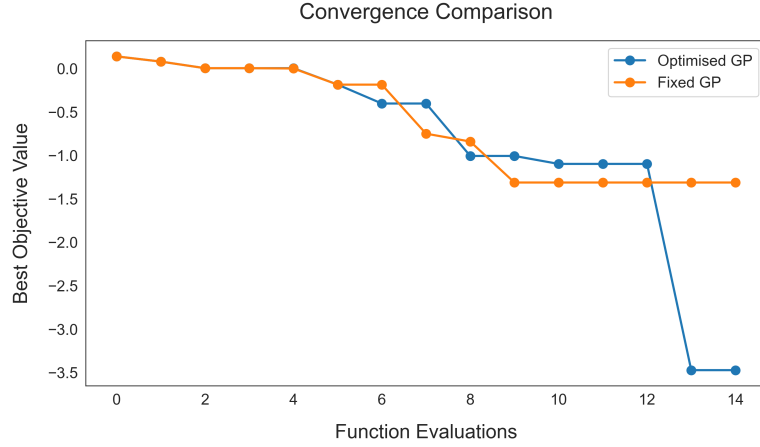


Figure 6.10: The Optimised GP found better solutions compared to the Fixed GP (-3.472 vs -1.312) but took longer (6.579s vs 5.565s).

This analysis underscores the importance of balancing speed and accuracy when deploying Bayesian optimisation. To manage this trade-off, users should tailor the frequency of hyperparameter updates based on computational constraints and desired precision. For functions with expensive evaluations, hyperparameters should be adapted more frequently to refine accuracy; conversely, prioritising speed necessitates less frequent updates.

### 6.3.2 Joint Hyperparameter Optimisation for a GP

In this example, we aim to optimise the two hyperparameters of an RBF kernel (2.2.1) in a GP to model synthetic data. The objective function  $\mathcal{L}(\boldsymbol{\theta})$  corresponds to the NLL of the GP given the data, which we minimise using a surrogate GP whose hyperparameters are optimised via gradient descent. This demonstrates Bayesian Optimisation’s ability to reframe optimisation problems, regardless of their original dimensionality, as surrogate hyperparameter tuning tasks. Here, we use a 2D case for conceptual clarity and visual interpretability, with  $\boldsymbol{\theta} = (\ell_{(\text{surrogate})}, \sigma_{f(\text{surrogate})})$  representing the surrogate GP hyperparameters and  $\boldsymbol{x} = (\ell_{(\text{main})}, \sigma_{f(\text{main})})$  denoting the main GP hyperparameters. While this example focuses on 2D mappings, the framework generalises to higher dimensions.

Following Algorithm 1, the loss function is defined as the NLL of  $\boldsymbol{x}$  evaluated at specific  $\ell$  and  $\sigma_f$  configurations. At each iteration, a candidate  $\boldsymbol{x}_t$  is assessed via NLL, while the EI acquisition function guides hyperparameter selection. Figures B.1 and B.2 (Appendix B) illustrate the evolution of NLL and EI landscapes for optimised and unoptimised Bayesian optimisation (BO) approaches, respectively. The optimised BO strategy systematically adjusts kernel hyperparam-

ters, prioritising local refinements in later iterations, whereas the unoptimised approach maintains a diffuse exploration pattern (Fig. 6.11).

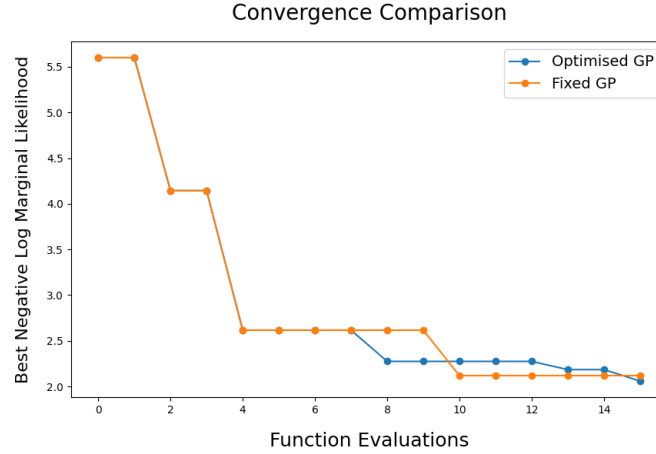


Figure 6.11: Convergence trends for joint hyperparameter optimisation. The optimised algorithm demonstrates adaptive search behaviour, transitioning from global exploration to local exploitation, while the unoptimised approach shows less systematic improvement.

The EI landscape evolution reveals a critical distinction: the optimised method progressively focuses on promising regions of the hyperparameter space, evidenced by increasingly localised EI maxima in successive iterations (Fig. B.1). This adaptivity enables efficient identification of optimal configurations, particularly when marginal likelihood gains significantly enhance model quality. Such behaviour underscores the advantage of our Adaptive Bayesian Optimisation framework in high-dimensional tasks, where complex parameter interactions complicate traditional search processes.

The code for this chapter and Algorithm 1 can be found in Appendix A.5 for reference.

# Chapter 7

## Conclusion

### 7.1 Summary of Findings

In this dissertation, we advanced the application of GPs in machine learning by addressing key aspects of regression, classification, and optimisation. We demonstrated how kernel choice significantly affects GP regression performance, underscoring the importance of selecting appropriate kernels. We also noted the limitation of GPs to predict well in regions for which there is little training data. In classification, we implemented and compared the Laplace approximation method with standard baseline techniques in the Heart Disease dataset, providing insight into the overall performance of these two models. By investigating various training point selection strategies (including LHS and ALM) we offered practical guidelines for optimising GP models. Our development of an improved adaptive Bayesian optimisation framework showed enhanced performance in hyperparameter tuning tasks compared to existing methods. These contributions collectively enhance GP methodologies, offering valuable insights for future research and applications across diverse domains.

### 7.2 Future Work

To conclude the report, we will suggest directions in which our work could be further developed.

- The snowline regression dataset focused on a Scottish dataset which was inherently noisy. It would be interesting to reattempt the regression process but for snowline data from countries with better meteorological recording infrastructure, for example, Switzerland. We also mentioned that GPs would not be useful in predicting snowline outwith Scotland due to the absence of data. We rephrased the problem to deal with this issue, and observed an improvement using GPs over a meteorological model. It would be interesting to take this further; GPs could also have potential in Avalanche

risk maps and prediction through modelling of snowline as in this report, or perhaps snow depth and compaction. This could prove an invaluable tool for the winter sports industry.

- In the classification chapter it would be interesting to further examine different approximation techniques. In particular, if we applied techniques such as Expectation Propagation, would we have been able to improve on the baseline even further.
- It is also worth considering that we need to pick a kernel ourselves. Our assumptions are often wrong, or the data is simply too complex. Several kernels can be compared to find the optimal kernel from a proposed class; however, there could be a much better kernel that we did not account for. Perhaps there is a way of learning the kernel from the dataset itself. This would be an interesting further research direction.
- We applied our novel Bayesian Optimisation approach to synthetic datasets and observed a notable improvement over existing approaches. It would be interesting to apply the method in a real-world scenario. For example, determining the best sites to drill for oil, or constructing seismometers in the best locations to detect earthquakes. These examples could take advantage of our method to reduce the impact of the expensive sampling procedure.

# Bibliography

- [1] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. Cambridge, MA, 2006. ISBN: 9780262182539. URL: <http://www.gaussianprocess.org/gpml/>.
- [2] Weizhong Yan, Hai Qiu, and Ya Xue. “Gaussian process for long-term time-series forecasting”. In: *2009 International Joint Conference on Neural Networks*. 2009.
- [3] Dongbing Gu and Huosheng Hu. “Spatial Gaussian Process Regression With Mobile Sensor Networks”. In: *IEEE Transactions on Neural Networks and Learning Systems* 23.8 (2012).
- [4] Roger A. Horn. *Matrix Analysis*. 1985.
- [5] Simon J.D. Prince. *Computer vision: models, learning and inference*. 2012.
- [6] D Duvenaud. “The Kernel Cookbook”. In: *University of Toronto* (). URL: <https://www.cs.toronto.edu/~duvenaud/cookbook>.
- [7] John Shawe-Taylor Nello Cristianini. *Kernel Methods for Pattern Analysis*. 2004.
- [8] Frank Bowman. *Introduction to Bessel Functions*. 2003.
- [9] Rasmussen Williams C. K. I. “Gaussian Processes for Regression”. In: *Advances in Neural Information Processing Systems 8 (NIPS 1995)* (1996). URL: [https://papers.nips.cc/paper\\_files/paper/1995/file/7cce53cf90577442771720a370c3c723-Paper.pdf](https://papers.nips.cc/paper_files/paper/1995/file/7cce53cf90577442771720a370c3c723-Paper.pdf).
- [10] A Gretton. *Introduction to Reproducing Kernel Hilbert Spaces (RKHS)*. 2019. URL: [https://www.gatsby.ucl.ac.uk/~gretton/coursefiles/lecture4\\_introToRKHS.pdf](https://www.gatsby.ucl.ac.uk/~gretton/coursefiles/lecture4_introToRKHS.pdf).
- [11] Rakan Nimer. *Air Passengers*. Kaggle. URL: <https://www.kaggle.com/datasets/rakannimer/air-passengers>.
- [12] Charles F. Van Loan Gene H. Golub. *Matrix Computations (3rd Edition)*. 1983.
- [13] M. Spencer. *Transcribed data for Scotland, 1945 to 2007*. NERC Environmental Information Data Centre. 2016. URL: <https://doi.org/10.5285/caf989a5-82d7-4db7-b6ff-c0475fdae07e>.
- [14] James R. Holton. *An Introduction to Dynamic Meteorology*. 2004.

- [15] Jr. Barbara J. Finlayson-Pitts James N. Pitts. *Chemistry of the Upper and Lower Atmosphere*. 2000.
- [16] Kirsty McCabe. *Rain, Sleet or Snow?* Royal Meteorological Society. 2022. URL: <https://www.rmets.org/metmatters/rain-sleet-or-snow>.
- [17] Yunji Chen. *AI Computing Systems, An Application-Driven Perspective*. 2023.
- [18] Txus Blasco. “A survey on uncertainty quantification in deep learning for financial time series prediction”. In: *Neurocomputing* (2024).
- [19] JP Dedieu. *Newton-Raphson Method*. 2015.
- [20] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. 2006.
- [21] David Lapp. *Heart Disease Dataset*. Originally from UCI Machine Learning Repository. 2017. URL: <https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset/data>.
- [22] C. Beulah Christalin Latha. “Improving the accuracy of prediction of heart disease risk based on ensemble classification techniques”. In: *Informatics in Medicine Unlocked* (2019).
- [23] Hercules Dalianis. *Evaluation Metrics and Evaluation*. 2018.
- [24] Sambu Seo et al. “Gaussian Process Regression: Active Data Selection and Test Point Rejection”. In: *Mustererkennung 2000*. Ed. by Gerald Sommer, Norbert Krüger, and Christian Perwass. Berlin, Heidelberg, 2000.
- [25] Jerome Sacks et al. “Design and Analysis of Computer Experiments”. In: *Statistical Science* 4.4 (1989).
- [26] M. D. McKay, R. J. Beckman, and W. J. Conover. “A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code”. In: *Technometrics* 21.2 (1979). URL: <https://doi.org/10.1080/00401706.1979.10489755>.
- [27] I. M. Sobol. “On the distribution of points in a cube and the approximate evaluation of integrals”. In: *Computational Mathematics and Mathematical Physics* 7.4 (1967).
- [28] Dae-Woo Kim et al. “Hybridization Algorithm of Fireworks Optimization and Generating Set Search for Optimal Design of IPMSM”. In: *IEEE Transactions on Magnetics* 53.6 (2017).
- [29] Robert B. Gramacy. *Surrogates: Gaussian process modeling, design, and optimization for the applied sciences*. eng. Chapman & Hall/CRC texts in statistical science series. Boca Raton, FL, 2020. ISBN: 9780367415426 9781000766523 9781000766202 9781000766363.
- [30] Richard H. Byrd et al. “A Limited Memory Algorithm for Bound Constrained Optimization”. In: *SIAM Journal on Scientific Computing* 16.5 (1995).
- [31] Vu Nguyen. “Bayesian Optimization for Accelerating Hyper-Parameter Tuning”. In: *2019 IEEE Second International Conference on Artificial Intelligence and Knowledge Engineering (AIKE)*. 2019.

- [32] Andrew Gordon Wilson. *Lecture 20: Gaussian Processes*. Carnegie Mellon University, 10-708: Probabilistic Graphical Models. 2015. URL: [https://www.cs.cmu.edu/~epxing/Class/10708-15/notes/10708\\_scribe\\_lecture20.pdf](https://www.cs.cmu.edu/~epxing/Class/10708-15/notes/10708_scribe_lecture20.pdf).
- [33] Michael S Pedersen Kaare B Petersen. *Matrix Cookbook*. 2007. URL: <https://www.math.uwaterloo.ca/~hwolkowi/matrixcookbook.pdf>.
- [34] James Bergstra, Daniel Yamins, and David D Cox. “Hyperparameter optimization: A review of algorithms and applications”. In: *Computational Intelligence* 31.3 (2015).
- [35] Donald R. Jones, Matthias Schonlau, and William J. Welch. “Efficient global optimization of expensive black-box functions”. In: *Journal of Global Optimization* 13.4 (1998).
- [36] Jasper Snoek, Hugo Larochelle, and Ryan P Adams. “Practical Bayesian Optimization of Machine Learning Algorithms”. In: *Advances in Neural Information Processing Systems*. Ed. by F. Pereira et al. Vol. 25. 2012. URL: [https://proceedings.neurips.cc/paper\\_files/paper/2012/file/05311655a15b75fab86956663e1819cd-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2012/file/05311655a15b75fab86956663e1819cd-Paper.pdf).
- [37] A. P. Dempster, N. M. Laird, and D. B. Rubin. “Maximum Likelihood from Incomplete Data via the EM Algorithm”. In: *Journal of the Royal Statistical Society. Series B (Methodological)* 39.1 (1977). URL: <http://www.jstor.org/stable/2984875>.
- [38] Bobak Shahriari et al. “Taking the Human Out of the Loop: A Review of Bayesian Optimization”. In: *Proceedings of the IEEE* 104.1 (2016).



# Appendix A

## Code

### A.1 Creating a Custom Kernel Code

Listing A.1: Python code for implementing and fitting the customised and Matérn kernel in section 2.2.3 to the air passenger dataset.

```
1 # Imports basic libraries
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6 # Import the modelling libraries
7 import GPy
8 from sklearn.preprocessing import StandardScaler
9
10 # Read the csv file (see bibliography for the source of the
   data)
11 df = pd.read_csv('AirPassengers.csv', parse_dates=['Month',
   ], index_col='Month')
12
13 # Converts the index to a numpy array of numerical
   timestamps and sets the training data vectors
14 X_train = df.index.to_numpy().astype('int64')
15 y_train = np.array(df['#Passengers'])
16
17 # Reshapes the training data for use in sklearn modules
18 X_train = X_train.reshape(-1, 1)
19 y_train = y_train.reshape(-1, 1)
20
21 # Scale X_train by subtracting the mean and dividing by the
   standard deviation
22 x_scaler = StandardScaler()
23 X_train_scaled = x_scaler.fit_transform(X_train)
24
```

```

25 # Scale y_train by subtracting the mean and dividing by the
   standard deviation
26 y_scaler = StandardScaler()
27 y_train_scaled = y_scaler.fit_transform(y_train)
28
29 # For the following kernels, the initial hyperparameter
   values are not highly significant; they will be
   optimised over
30 # Defines a GPy periodic kernel
31 k_periodic = GPy.kern.PeriodicExponential(input_dim=1,
   variance=0.3, period=0.1, lengthscale=0.1)
32 # Defines a GPy noise kernel
33 k_rand_noise = GPy.kern.White(input_dim=1, variance=0.1)
34 # Defines a GPy RBF kernel
35 k_exp = GPy.kern.RBF(input_dim=1, variance=0.01,
   lengthscale=0.5)
36 # Defines a GPy polynomial kernel
37 k_growth = GPy.kern.Poly(input_dim=1, order=2)
38
39 # Create the custom kernel described in section 2.2.3
40 kernel_custom = k_periodic * k_growth + k_exp +
   k_rand_noise
41
42 # Create the GP model that uses the custom kernel defined
   above (we scale translate the scaled data to allow the
   polynomial growth component
43 # to be modelled more reasonably.
44 custom_gp = GPy.models.GPRegression(X_train_scaled + 1.3,
   y_train_scaled, kernel_custom)
45
46 # Optimise the hyperparameters of the custom_gp model
47 custom_gp.optimize(optimizer='cg', max_iters=2000)
48
49 # Returns the optimised GP model
50 custom_gp
51
52
53 # For the following kernels, the initial hyperparameter
   values are not highly significant; they will be
   optimised over
54 # Defines a GPy polynomial kernel
55 k_mat = GPy.kern.Matern32(input_dim=1, variance=1,
   lengthscale=0.1)
56
57 # Create the GP model that uses the custom kernel defined
   above (we scale translate the scaled data to allow the
   polynomial growth component
58 # to be modelled more reasonably.
59 mat_gp = GPy.models.GPRegression(X_train_scaled + 1.3,
   y_train_scaled, k_mat)

```

```

60
61 # Optimise the hyperparameters of the custom_gp model
62 mat_gp.optimize(optimizer='cg', max_iters=2000)
63
64 # Returns the optimised GP model
65 mat_gp
66
67
68 # Plot results
69 # Define the test vector for the scaled and translated data
70 X_test_scaled = np.linspace(-0.5, 4.4, 100).reshape(-1, 1)
71
72 # Make predictions on X_test using the custom_gp model (GP
   posterior mean and variance)
73 mean_scaled, var_scaled = custom_gp.predict(X_test_scaled)
74
75 # Reverse the scaling for the mean predictions
76 mean = y_scaler.inverse_transform(mean_scaled)
77
78 # Reverse the translation applied and then reverse the
   scaling for X_test
79 X_test = x_scaler.inverse_transform(X_test_scaled-1.3)
80
81 # Flattens X_test for plotting and converts it back to date
   -time format
82 X_test = X_test.flatten()
83 X_test = pd.to_datetime(X_test, unit='ns')
84
85 # Reverse the scaling for the variance of the predictions
86 var = y_scaler.inverse_transform(var_scaled)
87 # Calculate standard deviation of the non-scaled
   predictions
88 std_dev = np.sqrt(var)
89
90
91 # Plot the predictions of the model and the variance of
   these predictions
92 plt.figure(figsize=(10, 6))
93 plt.plot(X_test, mean, 'b--', linewidth=2, label='Posterior
   Mean')
94 plt.fill_between(X_test, mean.flatten() - 1.96 * std_dev.
   flatten(),
95                  mean.flatten() + 1.96 * std_dev.flatten(),
   color='blue', alpha=0.2, label=r'$\pm$
   2 ')
96
97 # Plot the training data
98 plt.scatter(df.index, np.array(df['#Passengers']), color='
   red', marker='o', label='Training Data', s=15, alpha
   =0.8)

```

```

99
100 # Set labels and tick sizes
101 plt.xlabel('Year', fontsize=13)
102 plt.ylabel('Number of Passengers', fontsize=13)
103 plt.xticks(fontsize=12)
104 plt.yticks(fontsize=12)
105
106 # Display the legend, set title, set grid, and plot
107 plt.legend(fontsize=13)
108 plt.title('Air Passenger Dataset GP Prediction (Custom
    Kernel)', fontsize=16)
109 plt.grid(True, linestyle="--", alpha=0.6)
110 plt.savefig('Custom_kernel.png', dpi=500)
111 plt.show()
112
113
114 # Make predictions on X_test using the custom_gp model (GP
    posterior mean and variance)
115 mean_scaled, var_scaled = mat_gp.predict(X_test_scaled)
116
117 # Reverse the scaling for the mean predictions
118 mean = y_scaler.inverse_transform(mean_scaled)
119
120 # Reverse the scaling for the variance of the predictions
121 var = y_scaler.inverse_transform(var_scaled)
122 # Calculate standard deviation of the non-scaled
    predictions
123 std_dev = np.sqrt(var)
124
125
126 # Plot the predictions of the model and the variance of
    these predictions
127 plt.figure(figsize=(10, 6))
128 plt.plot(X_test, mean, 'b--', linewidth=2, label='Posterior
    Mean')
129 plt.fill_between(X_test, mean.flatten() - 1.96 * std_dev.
    flatten(),
130                  mean.flatten() + 1.96 * std_dev.flatten(),
    color='blue', alpha=0.2, label=r'$\pm$
    2 ')
131
132 # Plot the training data
133 plt.scatter(df.index, np.array(df['#Passengers']), color='
    red', marker='o', label='Training Data', s=15, alpha
    =0.8)
134
135 # Set labels and tick sizes
136 plt.xlabel('Year', fontsize=15)
137 plt.ylabel('Number of Passengers', fontsize=15)
138 plt.xticks(fontsize=14)

```

```

139 plt.yticks(fontsize=14)
140
141 # Display the legend, set title, set grid, and plot
142 plt.legend(fontsize=13)
143 plt.title('Air Passenger Dataset GP Prediction (Mat r n
      Kernel)', fontsize=20)
144 plt.grid(True, linestyle="--", alpha=0.6)
145 plt.savefig('Matern_kernel.png', dpi=500)
146 plt.show()

```

## A.2 Snowline Regression Analysis Code

Listing A.2: Python code for snowline prediction (Part 1 of Example)

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from mpl_toolkits.mplot3d import Axes3D
4 import csv
5 import math
6 from datetime import datetime
7 from scipy.linalg import cho_factor, cho_solve
8 import GPy
9 import os
10
11 # create dictionary of ${station_name}: {east:___, north:
    ___}
12 stations = {}
13 file = open("data/data/supporting-documents/Station_details
    .txt", "r")
14 file.readline() #header
15 for line in file.readlines():
16     line = line.split("|")
17     stations[line[0]] = {"east":line[2], "north":line[3]}
18 file.close()
19
20 # add data:[{date:___, snowline:___, snowelev:___}] to
    stations
21 directory_path = "./data/data/data"
22 file_extension = '.txt'
23 substring = 'SSGB_st_'
24
25 for filename in os.listdir(directory_path):
26     file_path = os.path.join(directory_path, filename)
27     if os.path.isfile(file_path) and filename.endswith(
        file_extension) and substring in filename:
28         with open(file_path, 'r') as file:
29             station_name=filename[8:-4]
30             file.readline() #header
31             data = []

```

```

32         for line in file.readlines():
33             line = line.split("|")
34             snowline = 0.0 if line[3][: -1] == "n"
35                     else float(line[3][: -1])
36             snowline = 0.0 if line[2] == "n" else (
37                 snowline if line[2] == "st" else float(
38                     line[2]))
39             data.append({"date": line[0], "snowline":
40                         snowline})
41         if station_name in stations:
42             stations[station_name]["data"] = data
43
44     # Return the number of days since the year 0
45     def timestamp(date_string):
46         is_leap = lambda year: year % 4 == 0 and (year % 100 !=
47             0 or year % 400 == 0)
48         date = datetime.strptime(date_string, '%Y-%m-%d')
49         total_days = sum(366 if is_leap(year) else 365 for year
50             in range(0, date.year))
51         total_days += (date - datetime(date.year, 1, 1)).days
52         return total_days
53
54     # Return data for a given date
55     def get_X_y_on_date(date):
56         date = "1970-12-31"
57         x = []
58         y = []
59         for station_name in stations:
60             station = stations[station_name]
61             for entry in station["data"]:
62                 if entry["date"] == date:
63                     snowline = entry["snowline"]
64                     if snowline > 0.0:
65                         x.append([float(station["east"]), float(
66                             station["north"])]])
67                         y.append([float(entry["snowline"])]])
68         return np.array(x), np.array(y)
69
70     X_whole, y_whole = get_X_y_on_date("1970-12-31")
71
72     # normalise and split data into train / test
73     eastings = np.array([X_whole[i][0] for i in range(len(
74         X_whole))])
75     northings = np.array([X_whole[i][1] for i in range(len(
76         X_whole))])
77     eastings_mean, eastings_std = eastings.mean(), eastings.std(
78         )
79     northings_mean, northings_std = northings.mean(), northings
80         .std()

```

```

71 eastings = (eastings - eastings_mean) / eastings_std
72 northings = (northings - northings_mean) / northings_std
73 y_train_mean, y_train_std = y_whole.mean(), y_whole.std()
74 y_whole = (y_whole - y_train_mean) / y_train_std
75
76 split = 0.75
77 idx = int((len(eastings)-1)*0.75)
78 eastings_test = eastings[idx:]
79 eastings_train = eastings[:idx]
80 northings_test = northings[idx:]
81 northings_train = northings[:idx]
82 y_test = y_whole[idx:]
83 y_train = y_whole[:idx]
84
85
86
87 # Plot the eastings and northings in 2d (both train and
    test datasets):
88
89 labels = [[265709, 845669, "Inverness"],
90           [263465, 784904, "Dalwhinnie"],
91           [311577, 723931, "Perth"],
92           [326150, 673046, "Edinburgh"],
93           [259528, 665307, "Glasgow"],
94           [394396, 807392, "Aberdeen"],
95           [183320, 728235, "Oban"],
96           [213118, 894057, "Ullapool"]]
97 for i in range(len(labels)):
98     labels[i] = [(labels[i][0]-eastings_mean)/ eastings_std
99                 , (labels[i][1] - northings_mean) / northings_std,
100                 labels[i][2]]
101
102 plt.scatter(eastings,northings, c=y_whole, cmap="gray")
103 plt.colorbar()
104 for label in labels:
105     plt.scatter(label[0], label[1], color="red")
106     plt.text(label[0], label[1], label[2], fontsize=13,
107             color="red", ha='left')
108
109 plt.title("Snow level (sites where snow is lying) for one
110         day", fontsize=16)
111 plt.xlabel("Eastings",fontsize=14)
112 plt.ylabel("Northings",fontsize=14)
113 plt.show()
114
115 # Perform GP Regression
116 min_east,max_east = min(eastings_train), max(eastings_train
117 )

```

```

114 min_north,max_north = min(northings_train),max(
    northings_train)
115
116 # Number of interpolated points
117 n_points = 50
118
119 # Generate linearly spaced values for eastings and
    northings
120 eastings_lin = np.linspace(min_east-0.5, max_east+0.5,
    n_points)
121 northings_lin = np.linspace(min_north-0.5, max_north+0.5,
    n_points)
122
123 # Create a grid of east-north pairs
124 east_grid, north_grid = np.meshgrid(eastings_lin,
    northings_lin)
125
126 # combine into a 1D array of (east, north) pairs
127 east_north_pairs = np.column_stack((east_grid.ravel(),
    north_grid.ravel()))
128
129 mean_pred, var_pred = model.predict(east_north_pairs)
130
131
132
133 mean_grid = mean_pred.reshape((n_points, n_points))
134 var_grid = var_pred.reshape((n_points, n_points))
135
136 # Plot the mean prediction as a contour
137 plt.figure(figsize=(12, 6))
138 plt.subplot(1, 2, 1)
139 plt.scatter(eastings_train,northings_train, c=y_train, cmap
    ="gray", label="train points")
140 contour_mean = plt.contour(east_grid, north_grid, mean_grid
    , cmap="Reds")
141 for label in labels:
142     plt.scatter(label[0], label[1], color="red")
143     plt.text(label[0], label[1], label[2], fontsize=8,
        color="red", ha='left')
144 plt.colorbar(contour_mean)
145 plt.title("Mean Prediction Contour")
146 plt.xlabel("Easting")
147 plt.ylabel("Northing")
148
149 #add the test points
150 plt.scatter(eastings_test,northings_test, c=y_test, cmap="
    gray", marker="x", label="test points")
151 plt.legend()
152
153 # Plot the variance prediction as a contour

```



```

154 plt.subplot(1, 2, 2)
155 plt.scatter(eastings_train, northings_train, c=y_train, cmap
            ="gray")
156 contour_var = plt.contour(east_grid, north_grid, var_grid,
            cmap="Blues")
157 for label in labels:
158     plt.scatter(label[0], label[1], color="red")
159     plt.text(label[0], label[1], label[2], fontsize=8,
            color="red", ha='left')
160 plt.colorbar(contour_var)
161 plt.title("Variance Prediction Contour")
162 plt.xlabel("Easting")
163 plt.ylabel("Northing")
164 plt.tight_layout()
165 plt.show()
166
167 # Summary statistics
168 def MSE(y_true, y_pred):
169     return np.mean((y_true - y_pred)**2)
170
171 def RMSE(y_true, y_pred):
172     return np.sqrt(MSE(y_true, y_pred))
173
174 def SumSquaredError(y_true, y_pred):
175     return np.sum((y_true - y_pred)**2)
176
177 def SummaryStats(y_true, y_pred):
178     return {
179         "MSE": MSE(y_true, y_pred),
180         "RMSE": RMSE(y_true, y_pred),
181         "SumSquaredError": SumSquaredError(y_true, y_pred)
182     }
183 test_east_north_pairs = np.column_stack((eastings_test,
            northings_test))
184 mean_pred_test, var_pred_test = model.predict(
            test_east_north_pairs)
185 residuals_test = y_test - mean_pred_test
186
187 mean_pred_train, var_pred_train = model.predict(np.
            column_stack((eastings_train, northings_train)))
188 residuals_train = y_train - mean_pred_train
189
190 stats = SummaryStats(y_train, mean_pred_train)
191 print("TRAIN", stats)
192 stats = SummaryStats(y_test, mean_pred_test)
193 print("TEST", stats)
194
195 # Plot 1: Train residuals vs predicted values
196 axes[0, 0].scatter(mean_pred_train, residuals_train, c='b',
            alpha=0.5)

```

```

197 axes[0, 0].axhline(y=0, color='r', linestyle='--')
198 axes[0, 0].set_xlabel("Predicted values")
199 axes[0, 0].set_ylabel("Residuals")
200 axes[0, 0].set_title("TRAIN Residuals vs Predicted values")
201
202 # Plot 2: Train residuals vs true values
203 axes[0, 1].scatter(y_train, residuals_train, c='b', alpha
    =0.5)
204 axes[0, 1].axhline(y=0, color='r', linestyle='--')
205 axes[0, 1].set_xlabel("True y values")
206 axes[0, 1].set_ylabel("Residuals")
207 axes[0, 1].set_title("TRAIN Residuals vs True values")
208
209 # Plot 3: Test residuals vs predicted values
210 axes[1, 0].scatter(mean_pred_test, residuals_test, c='b',
    alpha=0.5)
211 axes[1, 0].axhline(y=0, color='r', linestyle='--')
212 axes[1, 0].set_xlabel("Predicted values")
213 axes[1, 0].set_ylabel("Residuals")
214 axes[1, 0].set_title("TEST Residuals vs Predicted values")
215
216 # Plot 4: Test residuals vs true values
217 axes[1, 1].scatter(y_test, residuals_test, c='b', alpha
    =0.5)
218 axes[1, 1].axhline(y=0, color='r', linestyle='--')
219 axes[1, 1].set_xlabel("True y values")
220 axes[1, 1].set_ylabel("Residuals")
221 axes[1, 1].set_title("TEST Residuals vs True values")

```

Listing A.3: Python code for kernel comparison

```

1 import GPpy
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from sklearn.metrics import mean_squared_error
5 import math
6
7 def get_trained_models(eastings, northings, y_train):
8     # Define base kernels (excluding Periodic)
9     base_kernels = {
10         "RBF": GPpy.kern.RBF(input_dim=2, variance=0.1,
11                             lengthscale=0.1),
12         "Polynomial": GPpy.kern.Poly(input_dim=2, order=10),
13         "Matern32": GPpy.kern.Matern32(input_dim=2)
14     }
15
16     # Generate unique combinations of kernel sums and
17     # products
18     kernels = {
19         "Matern": base_kernels["Matern32"],

```

```

18     "RBF": base_kernels["RBF"],
19     "Polynomial": base_kernels["Polynomial"],
20     "Matern + RBF": base_kernels["Matern32"] +
        base_kernels["RBF"],
21     "Matern + Polynomial": base_kernels["Matern32"] +
        base_kernels["Polynomial"],
22     "Matern * Polynomial": base_kernels["Matern32"] *
        base_kernels["Polynomial"],
23 }
24
25 X_train = np.column_stack((eastings, northings))
26 models = {}
27
28 for name, kernel in kernels.items():
29     model = GPy.models.GPRegression(X_train, y_train,
        kernel)
30     model.optimize(messages=False)
31     models[name] = model
32
33 return models
34
35 # Perform GP Regression
36 n_points = 50
37 eastings_lin = np.linspace(min(eastings_train) - 0.5, max(
        eastings_train) + 0.5, n_points)
38 northings_lin = np.linspace(min(northings_train) - 0.5, max(
        northings_train) + 0.5, n_points)
39
40 east_grid, north_grid = np.meshgrid(eastings_lin,
        northings_lin)
41 east_north_pairs = np.column_stack((east_grid.ravel(),
        north_grid.ravel()))
42
43 models = get_trained_models(eastings_train, northings_train
        , y_train)
44
45 # Get global min/max for consistent shading across plots
46 all_means = []
47 rmse_results = []
48 for name, model in models.items():
49     mean_pred, _ = model.predict(east_north_pairs)
50     all_means.append(mean_pred)
51
52 # Evaluate model performance
53 y_test_pred, _ = model.predict(np.column_stack((
        eastings_test, northings_test)))
54 rmse = mean_squared_error(y_test, y_test_pred, squared=
        False)
55 rmse_results.append((name, rmse))
56 print(f"RMSE for {name}: {rmse}")

```

```

57
58 # Sort RMSE results
59 rmse_results.sort(key=lambda x: x[1])
60
61 global_min = np.min(all_means)
62 global_max = np.max(all_means)
63
64 num_models = len(models)
65 n_rows = math.ceil(math.sqrt(num_models))
66 n_cols = math.ceil(num_models / n_rows)
67
68 plt.figure(figsize=(12, 12))
69
70 for i, (name, model) in enumerate(models.items(), 1):
71     mean_pred, var_pred = model.predict(east_north_pairs)
72     mean_grid = mean_pred.reshape((n_points, n_points))
73
74     plt.subplot(n_rows, n_cols, i)
75     plt.scatter(eastings_train, northings_train, c=y_train,
76               cmap="gray", label="train points")
77     contour_mean = plt.contourf(east_grid, north_grid,
78                               mean_grid, cmap="Greens", alpha=0.7, vmin=global_min
79                               , vmax=global_max)
80     cbar = plt.colorbar(contour_mean)
81     cbar.set_label("Normalised Snowline", fontsize=14)
82     plt.title(f"Mean Prediction - {name}", fontsize=18)
83     plt.xlabel("Easting", fontsize=14)
84     plt.ylabel("Northing", fontsize=14)
85
86 plt.tight_layout()
87 plt.show()

```

Listing A.4: Python code for comparison between GP and meteorological model

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.gaussian_process import
5     GaussianProcessRegressor
6 from sklearn.gaussian_process.kernels import Matern, RBF,
7     RationalQuadratic, ExpSineSquared, ConstantKernel as C
8 from sklearn.metrics import mean_squared_error
9
10 # Function to remove outliers using IQR for features
11 def remove_feature_outliers(X, y, feature_columns,
12                             lower_threshold=1.5, upper_threshold=1.5):
13     for i, feature in enumerate(feature_columns):
14         Q1 = np.percentile(X[:, i], 25) # 1st quartile
15         Q3 = np.percentile(X[:, i], 75) # 3rd quartile
16         IQR = Q3 - Q1

```

```

14     lower_bound = Q1 - lower_threshold * IQR
15     upper_bound = Q3 + upper_threshold * IQR
16
17     # Keep only data points within the IQR bounds
18     mask = (X[:, i] >= lower_bound) & (X[:, i] <=
19           upper_bound)
20     X, y = X[mask], y[mask]
21
22     return X, y
23
24 # Function to filter snowline values
25 def filter_snowline(X, y, lower_bound, upper_bound):
26     mask = (y >= lower_bound) & (y <= upper_bound)
27     return X[mask], y[mask]
28
29 X_variables = ["east_grid", "north_grid", "tasmin", "
30               rainfall", "elevation"]
31
32 X = np.array([[entry[variable] for variable in X_variables]
33               for entry in complete_dataset])
34 y = np.array([entry["snowline"] for entry in
35               complete_dataset])
36
37 # Set new snowline lower and upper bounds
38 snowline_lower_bound = 50     # Keep snowlines above 50m
39 snowline_upper_bound = 600    # Keep snowlines below 600m
40
41 # Filter snowline based on new bounds
42 X, y = filter_snowline(X, y, snowline_lower_bound,
43               snowline_upper_bound)
44
45 # Remove feature outliers using IQR
46 X, y = remove_feature_outliers(X, y, X_variables)
47
48 # Split the data into train and test sets
49 TEST_TRAIN_SPLIT = 0.75
50 X_train, X_test = X[:int(TEST_TRAIN_SPLIT * len(X))], X[int
51               (TEST_TRAIN_SPLIT * len(X)):]
52 y_train, y_test = y[:int(TEST_TRAIN_SPLIT * len(y))], y[int
53               (TEST_TRAIN_SPLIT * len(y)):]
54
55 # Standardise features
56 scaler = StandardScaler()
57 X_train_scaled = scaler.fit_transform(X_train)
58 X_test_scaled = scaler.transform(X_test)
59
60 # Define a set of kernels to test
61 kernel_options = {
62     "Mat rn ( =1.5)": C(1.0) * Matern(length_scale=1.0,
63           nu=1.5),
64     "Mat rn ( =2.5)": C(1.0) * Matern(length_scale=1.0,

```

```

        nu=2.5),
56     "RBF": C(1.0) * RBF(length_scale=1.0),
57     "Rational Quadratic": C(1.0) * RationalQuadratic(
        length_scale=1.0, alpha=1.0),
58     "Periodic": C(1.0) * ExpSineSquared(length_scale=1.0,
        periodicity=365.25),
59     "Matern + RBF": C(1.0) * (Matern(length_scale=1.0, nu
        =1.5) + RBF(length_scale=1.0)),
60     "Matern * RBF": C(1.0) * (Matern(length_scale=1.0, nu
        =1.5) * RBF(length_scale=1.0)),
61     "Matern + Rational Quadratic": C(1.0) * (Matern(
        length_scale=1.0, nu=1.5) + RationalQuadratic(
        length_scale=1.0, alpha=1.0)),
62     "Matern * Rational Quadratic": C(1.0) * (Matern(
        length_scale=1.0, nu=1.5) * RationalQuadratic(
        length_scale=1.0, alpha=1.0)),
63     "RBF + Periodic": C(1.0) * (RBF(length_scale=1.0) +
        ExpSineSquared(length_scale=1.0, periodicity=365.25)
        ),
64     "RBF * Periodic": C(1.0) * (RBF(length_scale=1.0) *
        ExpSineSquared(length_scale=1.0, periodicity=365.25)
        ),
65 }
66
67 # Store predictions for RMSE calculation later
68 predictions = {}
69
70 # Plotting setup
71 fig, axes = plt.subplots(nrows=4, ncols=4, figsize=(6, 15))
72 axes = axes.flatten()
73
74 # Iterate through each kernel, train the model, and
    visualise results
75 for i, (kernel_name, kernel) in enumerate(kernel_options.
    items()):
76     print(f"Training with kernel: {kernel_name}")
77
78     # Train the model
79     model = GaussianProcessRegressor(kernel=kernel,
        n_restarts_optimizer=10)
80     model.fit(X_train_scaled, y_train)
81
82     # Make predictions and store them
83     y_pred, y_var = model.predict(X_test_scaled, return_std
        =True)
84     predictions[kernel_name] = y_pred # Store predictions
        for RMSE computation
85
86     # Plot True vs Predicted
87     ax = axes[i]

```

```

88     ax.set_xlabel("True snowline (m)", fontsize=14)
89     ax.set_ylabel("Predicted snowline (m)", fontsize=14)
90     ax.set_title(f"Predictions from GP", fontsize=16)
91     ax.plot([0,620], [0,620], color='red', linestyle="--")
92     ax.scatter(y_test, y_pred, alpha=0.6)
93     ax.tick_params(axis='both', labelsize=12)
94     ax.set_xlim(0, 620) # Set x-axis range from 0 to 700
95     ax.set_ylim(0, 620) # Set y-axis range from 0 to 700
96
97 plt.tight_layout()
98 plt.show()
99
100 # Compute RMSE using stored predictions
101 def compute_rmse(predictions, y_test):
102     """Compute and print RMSE for each kernel using stored
103     predictions."""
104     print("\nKernel Type -- RMSE")
105     print("-" * 30)
106
107     for kernel_name, y_pred in predictions.items():
108         rmse = np.sqrt(mean_squared_error(y_test, y_pred))
109         print(f"{kernel_name:20} -- {rmse:.3f}")
110
111 compute_rmse(predictions, y_test)

```

## A.3 Heart Disease Application Code

Listing A.5: Python code for the Laplace based GP classification model

```

1 import GPy
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import pandas as pd
5 from sklearn.preprocessing import MinMaxScaler
6
7 # Load the training and test datasets respectively
8 df_train = pd.read_csv('heart_train_data.csv')
9 df_test = pd.read_csv('heart_test_data.csv')
10
11 # Sets the relevant feature and target names respectively
12 features = ["age", "thalach"]
13 target = "target"
14
15 # Consider only every 3rd data point
16 n = 3
17 X_train = df_train[features].iloc[0::n].values
18 X_test = df_test[features].iloc[0::n].values
19 y_train = df_train[target].iloc[0::n].values
20 y_test = df_test[target].iloc[0::n].values

```

```

21
22
23 # Apply the MinMax scaler to the data arrays to ensure
   predictors share a common scale
24 scaler = MinMaxScaler()
25 X_train_scaled = scaler.fit_transform(X_train) # Keep a
   scaled version of the training data
26 X_test_scaled = scaler.transform(X_test) # Keep a scaled
   version of the test data
27
28 # Defines a GPy rational quadratic kernel
29 k_rat = GPy.kern.RatQuad(input_dim=2, variance=1.0,
   lengthscales=1.0, power=1.5)
30 k_mat = GPy.kern.Matern32(input_dim=2, variance=1,
   lengthscales=0.1)
31
32 # Reshapes the training response vector to be compatible
   with GPy
33 y_train = y_train.reshape(-1, 1)
34
35 # Create GP classification model using the Laplace
   approximation
36 model = GPy.models.GPClassification(X_train_scaled, y_train
   , k_rat, inference_method=GPy.inference.
   latent_function_inference.Laplace())
37
38 # Optimise the hyperparameters of the model (using marginal
   likelihood)
39 model.optimize()
40
41 # Generate a grid for the contour plot (this grid is in the
   scaled input space)
42 xx, yy = np.meshgrid(np.linspace(-0.05, 1.05, 500), np.
   linspace(-0.05, 1.05, 500))
43 grid = np.c_[xx.ravel(), yy.ravel()]
44
45 # Apply an inverse transform to the grid to the original
   input space
46 grid_unscaled = scaler.inverse_transform(grid)
47
48 # Make predictions on the scaled test space
49 probs, _ = model.predict(grid)
50 probs = probs.reshape(xx.shape)
51
52 # Plot probabilities for the data in the unscaled space
53 plt.figure(figsize=(8, 6))
54 plt.contourf(grid_unscaled[:, 0].reshape(xx.shape),
   grid_unscaled[:, 1].reshape(yy.shape), probs, cmap='
   coolwarm', alpha=0.6)
55

```



```

56 # Add a colour bar
57 cbar = plt.colorbar()
58 cbar.set_label(label='Laplace Predictive Probability',
59               fontsize=13)
60
61 # Plot the decision boundary
62 plt.contour(grid_unscaled[:, 0].reshape(xx.shape),
63             grid_unscaled[:, 1].reshape(yy.shape),
64             probs, levels=[0.5], colors='black', linewidths
65             =2)
66
67 # Apply an inverse transform to the training data to the
68 # original input space
69 X_train_unscaled = scaler.inverse_transform(X_train_scaled)
70
71 # Identify the unique classes in y_train
72 classes = np.unique(y_train)
73
74 # List the colours we will use for the data classes and
75 # their labels
76 colors = ['blue', 'red']
77 class_name = ['No Disease', 'Disease']
78
79 # Scatter plot each class with their respective labels and
80 # colours
81 for i, class_label in enumerate(classes):
82     plt.scatter(X_train_unscaled[y_train.flatten() ==
83     class_label, 0],
84               X_train_unscaled[y_train.flatten() ==
85               class_label, 1],
86               label=class_name[i],
87               color=colors[i], edgecolors='k')
88
89 # Add labels and title
90 plt.xlabel('Age', fontsize=13)
91 plt.ylabel('thalach', fontsize=13)
92 plt.xticks(fontsize=13)
93 plt.yticks(fontsize=13)
94 plt.title("Laplace Decision Boundary", fontsize=16)
95
96 # Add the legend and display the plot
97 plt.legend()
98 plt.show()

```

Listing A.6: Python code for the Logistic regression model

```

1 # Imports relevant libraries for the GP classification
  # model
2 import numpy as np

```

```

3 import matplotlib.pyplot as plt
4 from sklearn.linear_model import LogisticRegression
5 from sklearn.preprocessing import MinMaxScaler
6 import pandas as pd
7
8 # Load the training and test datasets respectively
9 df_train = pd.read_csv('heart_train_data.csv')
10 df_test = pd.read_csv('heart_test_data.csv')
11
12 # Sets the relevant feature and target names respectively
13 features = ["age", "thalach"]
14 target = "target"
15
16 # Consider only every 3rd data point
17 n = 3
18 X_train = df_train[features].iloc[0::n].values
19 X_test = df_test[features].iloc[0::n].values
20 y_train = df_train[target].iloc[0::n].values
21 y_test = df_test[target].iloc[0::n].values
22
23 # Apply the MinMax scaler to the data arrays to ensure
    predictors share a common scale
24 scaler = MinMaxScaler()
25 X_train_scaled = scaler.fit_transform(X_train)
26 X_test = scaler.transform(X_test)
27
28 # Train a logistic model on the data
29 model = LogisticRegression()
30 model.fit(X_train_scaled, y_train)
31
32 # Generate a grid for the contour plot (this grid is in the
    scaled input space)
33 xx, yy = np.meshgrid(np.linspace(-0.05, 1.05, 500), np.
    linspace(-0.05, 1.05, 500))
34 grid = np.c_[xx.ravel(), yy.ravel()]
35
36 # Apply an inverse transform to the grid to the original
    input space
37 grid_unscaled = scaler.inverse_transform(grid)
38
39 # Make predictions on the scaled test space
40 probs = model.predict_proba(grid)[: , 1].reshape(xx.shape)
41
42 # Plot probabilities for the data in the unscaled space
43 plt.figure(figsize=(8, 6))
44 plt.contourf(grid_unscaled[: , 0].reshape(xx.shape),
    grid_unscaled[: , 1].reshape(yy.shape), probs, levels=np.
    linspace(0, 1, 11), cmap="coolwarm", alpha=0.8)
45
46 # Add a colour bar

```

```

47 cbar = plt.colorbar(label='Logistic Predictive Probability'
48 )
49 cbar.set_label(label='Logistic Predictive Probability',
50               fontsize=13)
51 # Plot the decsion boundary
52 plt.contour(grid_unscaled[:, 0].reshape(xx.shape),
53             grid_unscaled[:, 1].reshape(yy.shape),
54             probs, levels=[0.5], colors='black', linewidths
55             =2)
56 # Apply an inverse transform to the training data to the
57 # original input space
58 X_train_unscaled = scaler.inverse_transform(X_train_scaled)
59 # Identify the unique classes in y_train
60 classes = np.unique(y_train)
61 # List the colours we will use for the data classes and
62 # their labels
63 colors = ['blue', 'red']
64 class_name = ['No Disease', 'Disease']
65 # Scatter plot each class with their respective labels and
66 # colours
67 for i, class_label in enumerate(classes):
68     plt.scatter(X_train_unscaled[y_train.flatten() ==
69                               class_label, 0],
70               X_train_unscaled[y_train.flatten() ==
71                               class_label, 1],
72               label=class_name[i],
73               color=colors[i], edgecolors='k')
74 # Add labels and title
75 plt.xlabel('Age', fontsize=13)
76 plt.ylabel('thalach', fontsize=13)
77 plt.xticks(fontsize=13)
78 plt.yticks(fontsize=13)
79 plt.title("Logistic Model Decision Boundary", fontsize=16)
80 # Add the legend and display the plot
81 plt.legend()
82 plt.show()

```

## A.4 Active Learning Experiment

Listing A.7: Active Learning Experiment Code

```

1 import numpy as np

```

```

2 import pandas as pd
3 import warnings
4 from sklearn.gaussian_process import
    GaussianProcessRegressor
5 from sklearn.gaussian_process.kernels import Matern,
    WhiteKernel, ConstantKernel
6 from sklearn.metrics import mean_squared_error
7 from scipy.stats import ttest_rel, qmc
8
9 # Suppress convergence warnings
10 warnings.filterwarnings("ignore", message="The optimal
    value found for dimension")
11
12 # Benchmark functions
13 def branin(X):
14     x1 = X[:, 0]; x2 = X[:, 1]
15     a = 1.0; b = 5.1/(4*np.pi**2); c = 5.0/np.pi
16     r = 6.0; s = 10.0; t = 1.0/(8*np.pi)
17     return a*(x2 - b*x1**2 + c*x1 - r)**2 + s*(1-t)*np.cos(
        x1) + s
18
19 def six_hump_camel(X):
20     x1 = X[:, 0]; x2 = X[:, 1]
21     return (4*x1**2 - 2.1*x1**4 + (1/3)*x1**6 +
        x1*x2 - 4*x2**2 + 4*x2**4)
22
23
24 test_functions = {
25     "Branin": {"f": branin, "lb": [-5.0, 0.0], "ub": [10.0,
        15.0]},
26     "Six-Hump Camel": {"f": six_hump_camel, "lb": [-2.0,
        -1.2], "ub": [2.0, 1.2]}
27 }
28
29 # Sampling methods
30 def generate_random_points(N, lb, ub):
31     return np.random.uniform(lb, ub, (N, len(lb)))
32
33 def generate_lhs_points(N, lb, ub):
34     sampler = qmc.LatinHypercube(d=len(lb))
35     return qmc.scale(sampler.random(N), lb, ub)
36
37 def generate_sobol_points(N, lb, ub):
38     p = 2 ** int(np.ceil(np.log2(N)))
39     sampler = qmc.Sobol(d=len(lb), scramble=True)
40     return qmc.scale(sampler.random(p)[:N], lb, ub)
41
42 sampling_methods = {
43     "Random": generate_random_points,
44     "LHS": generate_lhs_points,
45     "Sobol": generate_sobol_points

```

```

46 }
47
48 # GP training and evaluation
49 def train_and_evaluate(X_train, y_train, X_test, y_test):
50     kernel = ConstantKernel(1.0, (1e-2, 1e3)) * Matern(
51         length_scale=1.0, nu=2.5) + \
52         WhiteKernel(noise_level=1e-5, (1e-8, 1e2))
53     gp = GaussianProcessRegressor(kernel=kernel,
54                                   n_restarts_optimizer=5)
55     gp.fit(X_train, y_train)
56     return np.sqrt(mean_squared_error(y_test, gp.predict(
57         X_test)))
58
59 # Active learning with MacKay strategy
60 def active_learning_mackay(func, lb, ub, initial_N, total_N
61 ):
62     X_train = generate_sobol_points(initial_N, lb, ub)
63     y_train = func(X_train)
64
65     for _ in range(total_N - initial_N):
66         gp = GaussianProcessRegressor(
67             kernel=ConstantKernel(1.0)*Matern(nu=2.5)+
68             WhiteKernel(),
69             n_restarts_optimizer=5
70         ).fit(X_train, y_train)
71         candidates = generate_random_points(1000, lb, ub)
72         next_point = candidates[np.argmax(gp.predict(
73             candidates, return_std=True)[1])]
74         X_train = np.vstack([X_train, next_point])
75         y_train = np.append(y_train, func(next_point.
76             reshape(1, -1)))
77
78     return X_train, y_train
79
80 # Experimental parameters
81 n_trials = 100; N_train = 15; initial_N = 5; N_test = 1000
82 results = []
83
84 for func_name, meta in test_functions.items():
85     f, lb, ub = meta["f"], meta["lb"], meta["ub"]
86     X_test = generate_random_points(N_test, lb, ub)
87     y_test = f(X_test)
88
89     for trial in range(n_trials):
90         # Traditional sampling methods
91         for method, sampler in sampling_methods.items():
92             X_train = sampler(N_train, lb, ub)
93             results.append({
94                 "Function": func_name,
95                 "Method": method,

```

```

89         "Trial": trial,
90         "RMSE": train_and_evaluate(X_train, f(
91             X_train), X_test, y_test)
92     })
93
94     # Active learning method
95     X_al, y_al = active_learning_mackay(f, lb, ub,
96         initial_N, N_train)
97     results.append({
98         "Function": func_name,
99         "Method": "Sobol+MacKay",
100        "Trial": trial,
101        "RMSE": train_and_evaluate(X_al, y_al, X_test,
102            y_test)
103    })
104
105 results_df = pd.DataFrame(results)
106
107 # Statistical analysis
108 print("Paired t-tests (RMSE comparisons):")
109 for func in results_df["Function"].unique():
110     func_data = results_df[results_df["Function"] == func]
111     active = func_data[func_data["Method"] == "Sobol+MacKay"]
112     baseline = func_data[func_data["Method"] == "Random"]
113     t, p = ttest_rel(active["RMSE"], baseline["RMSE"])
114     print(f"{func}: {method} vs Sobol+MacKay | t={t:.2f}, p={p:.2e}")

```

## A.5 Adaptive Bayesian Optimisation Code

Listing A.8: Python code for optimising two hyperparameters using Bayesian optimisation

```

1 import numpy as np
2 import GPy
3 from scipy.stats import norm
4 from scipy.spatial.distance import cdist
5
6 # Generate synthetic data
7 noise = 0.1
8 X_data = np.linspace(-5, 5, 50)[:, None]
9 Y_data = np.sin(X_data) + noise * np.random.randn(50, 1)
10
11 # Define the primary GP's objective function: log marginal
    likelihood

```

```

12 def evaluate_primary_gp(lengthscale, variance):
13     kernel = GPy.kern.RBF(input_dim=1, lengthscale=
        lengthscale, variance=variance)
14     gp_model = GPy.models.GPRegression(X_data, Y_data,
        kernel)
15     gp_model.Gaussian_noise.variance = 0.01 # Fixed noise
        variance
16     return gp_model.log_likelihood()
17
18 # Initial data point for the surrogate GP
19 initial_lengthscale = 1.0
20 initial_variance = 1.0
21 initial_log_marginal_likelihood = evaluate_primary_gp(
    initial_lengthscale, initial_variance)
22
23 # Surrogate GP data
24 X_surrogate = np.array([[initial_lengthscale,
    initial_variance]])
25 Y_surrogate = np.array([[initial_log_marginal_likelihood]])
26
27 # Define a flexible kernel for the surrogate GP with
    Automatic Relevance Determination (ARD)
28 kernel_surrogate = GPy.kern.RBF(input_dim=2, variance=1.0,
    lengthscale=np.array([1.0,
    1.0]), ARD=True)
29
30 kernel_surrogate.lengthscale.constrain_bounded(1e-2, 10.0)
31 kernel_surrogate.variance.constrain_bounded(1e-2, 10.0)
32
33 # Create and optimise the surrogate GP model
34 surrogate_gp = GPy.models.GPRegression(X_surrogate,
    Y_surrogate, kernel_surrogate)
35 surrogate_gp.Gaussian_noise.variance.constrain_bounded(1e
    -6, 1e-2)
36 surrogate_gp.optimise()
37
38 # Set of evaluated (lengthscale, variance) pairs
39 evaluated_points = X_surrogate.copy()
40 distance_threshold = 0.05 # Minimum distance between
    evaluated points
41
42 # Expected Improvement acquisition function for two-
    dimensional input
43 def expected_improvement(surrogate_gp, x_new, y_best):
44     mean, variance = surrogate_gp.predict(x_new)
45     std_dev = np.sqrt(variance)
46     # Avoid division by zero
47     std_dev = np.where(std_dev == 0, 1e-12, std_dev)
48     z = (mean - y_best) / std_dev
49     ei = (mean - y_best) * norm.cdf(z) + std_dev * norm.pdf
        (z)

```

```

50     return ei
51
52     # Function to suggest the next (lengthscale, variance) pair
53     def suggest_next_params(surrogate_gp, candidate_points,
54                             y_best, evaluated_points, distance_threshold):
55         # Compute distances from candidate points to evaluated
56         points
57         distances = cdist(candidate_points, evaluated_points)
58         min_distances = distances.min(axis=1)
59         # Filter out points that are too close to evaluated
60         points
61         valid_indices = np.where(min_distances >
62                                   distance_threshold)[0]
63         if len(valid_indices) == 0:
64             valid_indices = np.where(min_distances >= 0)[0] #
65             Consider all points if necessary
66         candidate_points_filtered = candidate_points[
67             valid_indices]
68         # Compute EI for all candidate points
69         ei_values = np.array([
70             expected_improvement(surrogate_gp, pt[None, :],
71                                   y_best)
72             for pt in candidate_points_filtered
73         ]).flatten()
74         # Suggest the next point with the highest EI
75         next_point = candidate_points_filtered[np.argmax(
76             ei_values)]
77         return next_point[0], next_point[1]
78
79     # Optimisation loop
80     max_iterations = 20
81     convergence_threshold = 1e-3
82     lengthscale_range = np.linspace(0.1, 5.0, 100)
83     variance_range = np.linspace(0.1, 5.0, 100)
84     y_best = initial_log_marginal_likelihood
85
86     # Create a grid of candidate points
87     lengthscale_grid, variance_grid = np.meshgrid(
88         lengthscale_range, variance_range)
89     candidate_points = np.vstack([lengthscale_grid.ravel(),
90                                   variance_grid.ravel()]).T
91
92     for iteration in range(max_iterations):
93         # Suggest the next (lengthscale, variance) pair
94         next_lengthscale, next_variance = suggest_next_params(
95             surrogate_gp, candidate_points, y_best,
96             evaluated_points, distance_threshold
97         )
98
99         # Evaluate the primary GP on the suggested parameters

```



```

89     current_log_marginal_likelihood = evaluate_primary_gp(
        next_lengthscale, next_variance)
90
91     # Update y_best if the new observation is better
92     if current_log_marginal_likelihood > y_best:
93         y_best = current_log_marginal_likelihood
94
95     # Add the new data to the surrogate GP's dataset
96     new_point = np.array([[next_lengthscale, next_variance
        ]])
97     X_surrogate = np.vstack([X_surrogate, new_point])
98     Y_surrogate = np.vstack([Y_surrogate, [
        current_log_marginal_likelihood]])
99     evaluated_points = np.vstack([evaluated_points,
        new_point])
100
101     # Update and optimise the surrogate GP with the new
        data point
102     surrogate_gp.set_XY(X_surrogate, Y_surrogate)
103     surrogate_gp.optimize()
104
105     # Check for convergence
106     if len(Y_surrogate) > 1 and abs(Y_surrogate[-1] -
        Y_surrogate[-2]) < convergence_threshold:
107         break
108
109     # The optimal hyperparameters found
110     optimal_lengthscale = X_surrogate[np.argmax(Y_surrogate[:,
        0]), 0]
111     optimal_variance = X_surrogate[np.argmax(Y_surrogate[:, 0])
        , 1]
112
113     # Fit the primary GP with optimal hyperparameters
114     optimal_kernel = GPy.kern.RBF(input_dim=1, lengthscale=
        optimal_lengthscale, variance=optimal_variance)
115     optimal_gp_model = GPy.models.GPRegression(X_data, Y_data,
        optimal_kernel)
116     optimal_gp_model.Gaussian_noise.variance = 0.01 # Fixed
        noise variance

```

# Appendix B

## Figures

### B.1 Adaptive Bayesian Optimisation 2D Updates

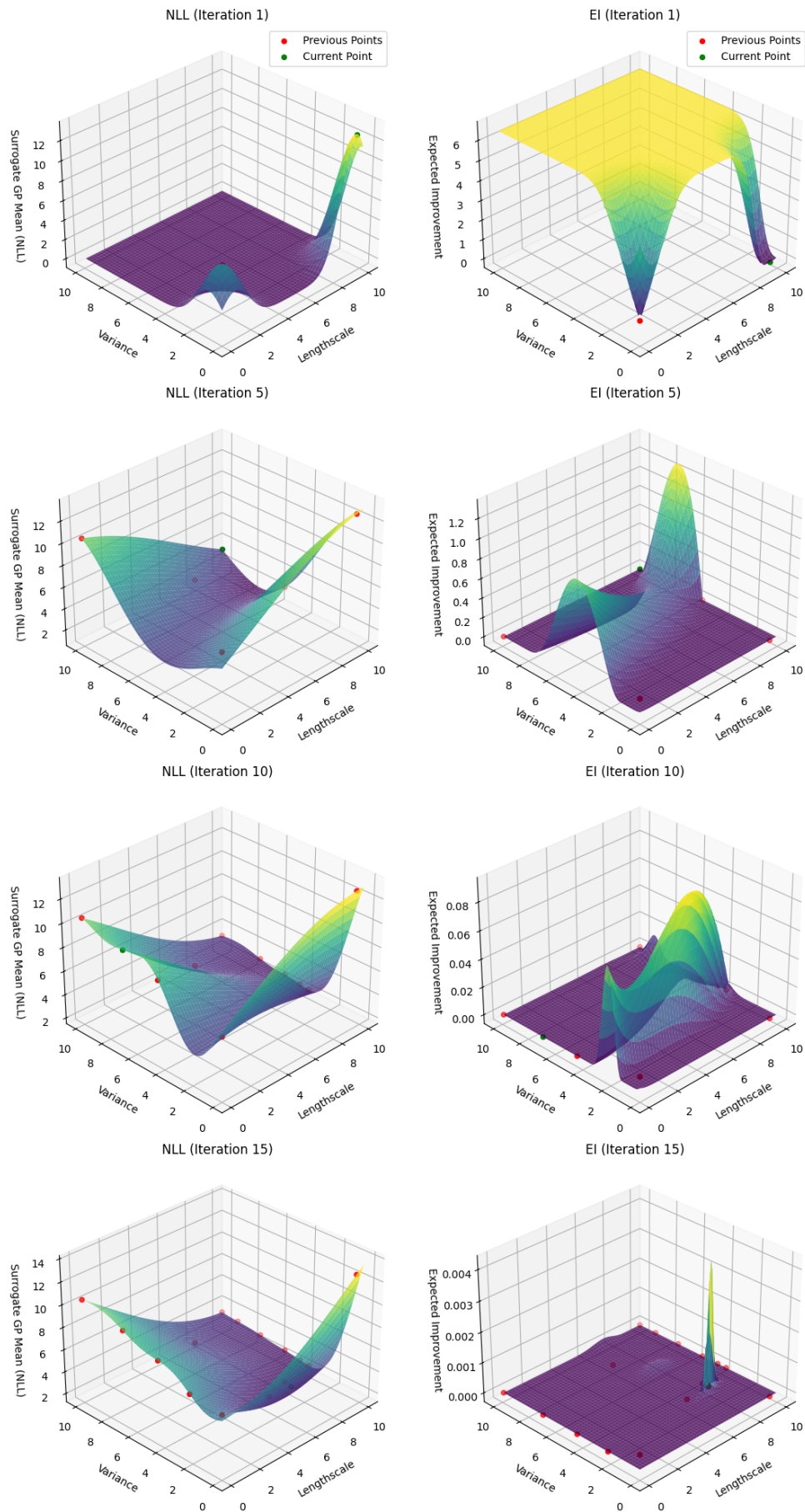


Figure B.1: Convergence of the optimised algorithm.

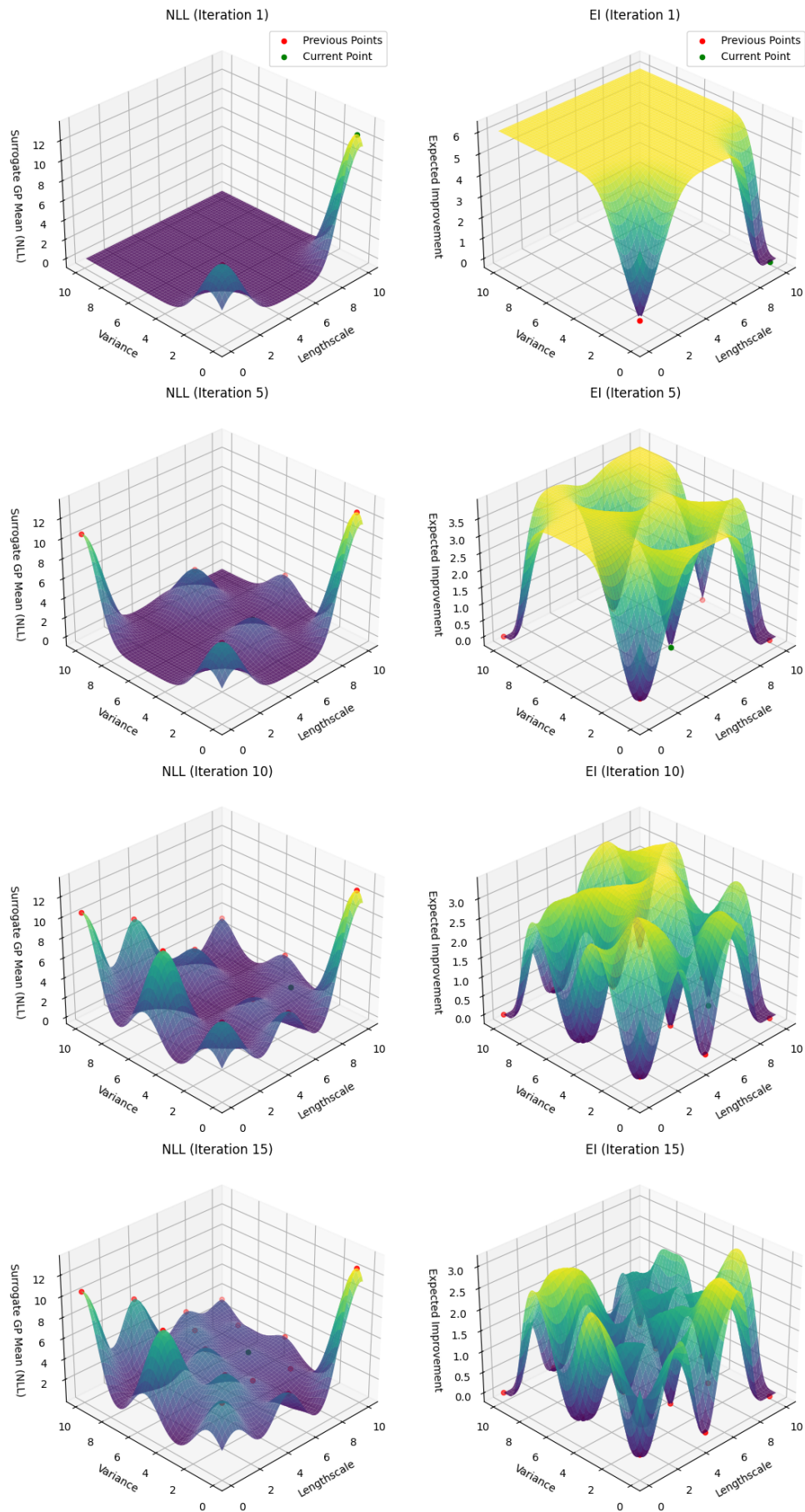


Figure B.2: Convergence of the unoptimised algorithm.